MTH00050 – Combinatorial Mathematics

Lecture 6: Graphs

Lecturer: Bùi Văn Thạch
Lab instructors: Nguyễn Ngọc Toàn, Trần Thị Thảo Nhi, Lê Đức Khoan
{bvthach,nntoan,tttnhi,ldkhoan}@fit.hcmus.edu.vn

(Tentative) Schedule 2026

No.	Date	Location	Topic
1	13/01 16/01	C24 C24/E302	Introduction, Sets, Proofs
2	20/01 23/01	C24 C24/E302	General counting methods
3	27/01 30/01	C24 C24/E302	Inclusion-exclusion principle
4	03/02 06/02	C24 C24/E302	Recurrence relations
5	03/03 06/03	C24 C24/E302	Generating functions
			Mid-term test

No.	Date	Location	Topic
6	10/03 13/03	C24 C24/E302	Graphs (Part I)
7	17/03 20/03	C24 C24/E302	Graphs (Part II)
8	24/03 27/03	C24 C24/E302	Path problems
9	31/03 03/04	C24 C24/E302	Tree problems
10	07/04 10/04	C24 C24/E302	Network flows (Part I)
11	14/04 17/04	C24 C24/E302	Network flows (Part II)
			Final exam

Course topics

0. Introduction
1. Set and counting
2. General counting methods for arrangements and selections
3. Inclusion-exclusion principle and Mobius inversion
4. Recurrence relations
5. Generating functions
6. Graphs
7. Tree problems
8. Path problems
9. Network flows

Goals

1. Be fluent in the language of graphs so you can analyze and model problems effectively.
   - Understand foundational terms: vertex, edge, degree, path, cycle, connectedness, adjacency.
   - Distinguish between types of graphs: directed, undirected, weighted, unweighted, cyclic, acyclic.
2. Choose the right data structure for a given graph problem to optimize performance.
   - Adjacency matrix, Adjacency list, Edge list.
3. Explore, analyze, and manipulate graph structures.
   - Depth-First Search (DFS),
   - Breadth-First Search (BFS).

Outline

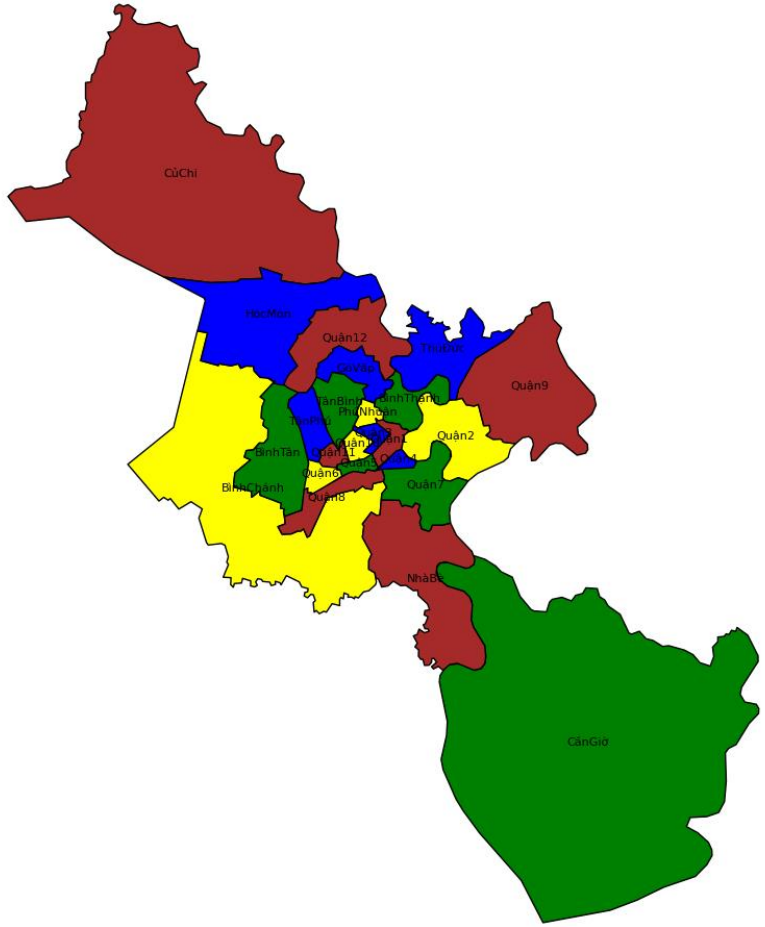
1. Introduction
2. Definition and basic properties
3. Trails, paths, and circuits
4. Graph representations
5. Isomorphisms
6. Graph traversals
   - Breadth-First Search (BFS)
   - Depth-First Search (DFS)

Outline

1. Introduction
2. Definition and basic properties
3. Trails, paths, and circuits
4. Graph representations
5. Isomorphisms
6. Graph traversals
   - Breadth-First Search (BFS)
   - Depth-First Search (DFS)

22 districts in Ho Chi Minh city

Shall we add some colors to this map of Ho Chi Minh City?



The Four-Color Theorem (1/)

- First proposed by Francis Guthrie in 1852 [1], the conjecture stated that:
  - Any map drawn on a plane can be colored using NO MORE THAN FOUR COLORS, in such a way that NO TWO adjacent regions SHARE the same color.
- Over the years, many incorrect proofs were attempted.
- In 1977, the theorem was **first rigorously proven by Appel and Haken** [2], with the aid of computer verification.
- A subsequent proof was later provided by Robertson and collaborators in 1996 [3].

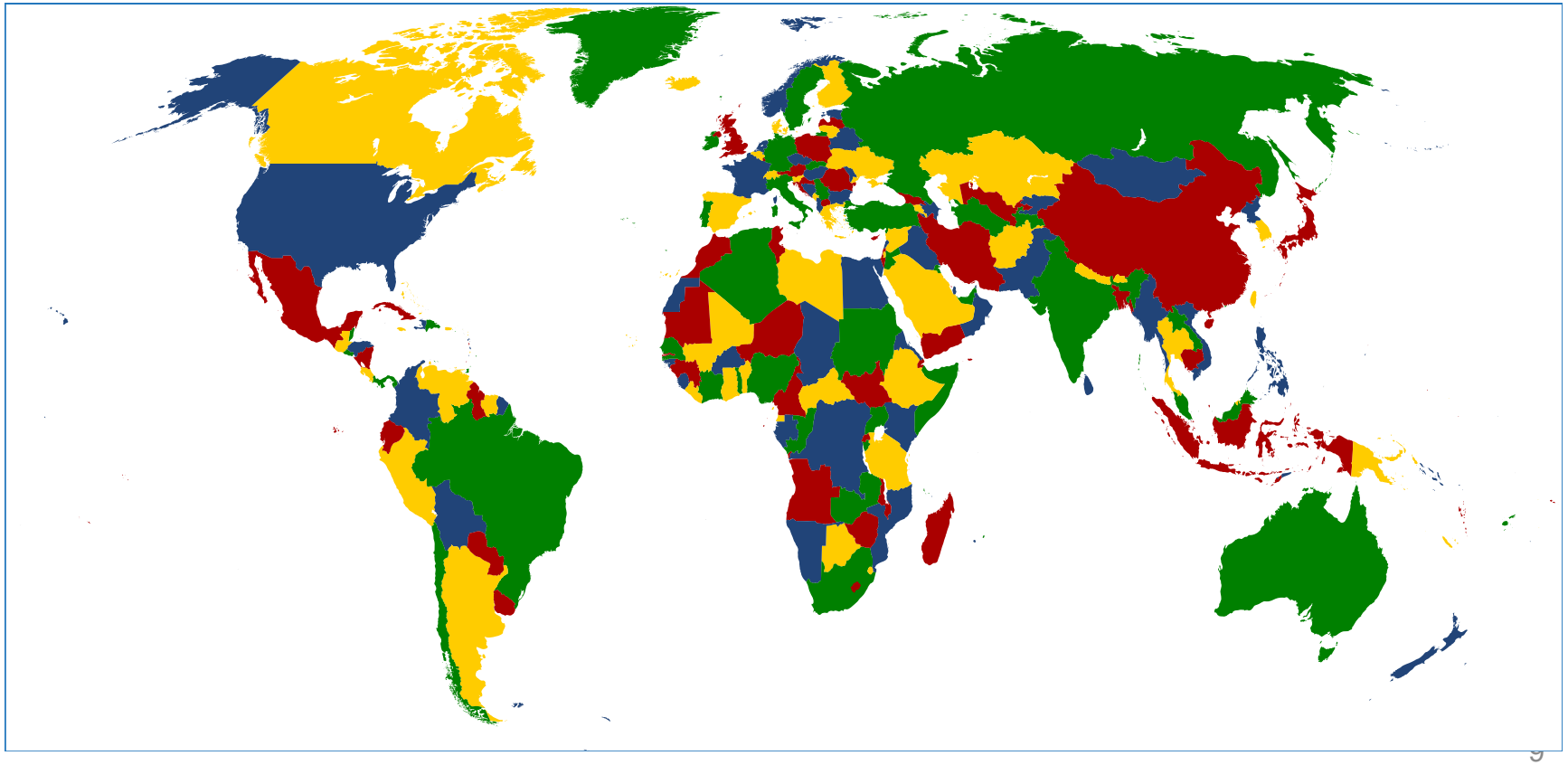
[1] May, Kenneth O. "The origin of the four-color conjecture." *Isis* 56.3 (1965): 346-348.
[2] Appel, Kenneth, and Wolfgang Haken. "The solution of the four-color-map problem." Scientific American 237.4 (1977): 108-121.
[3] Robertson, Neil, et al. "Efficiently four-coloring planar graphs." Proceedings of the 28th annual ACM STOC. 1996.

The Four-Color Theorem (2/)

Can we color the world map by countries with up to 4 colors such that

NO TWO adjacent countries SHARE the same color?

- But this is a map, not a graph!
- However, we can model it as a graph.
- But what is a graph?

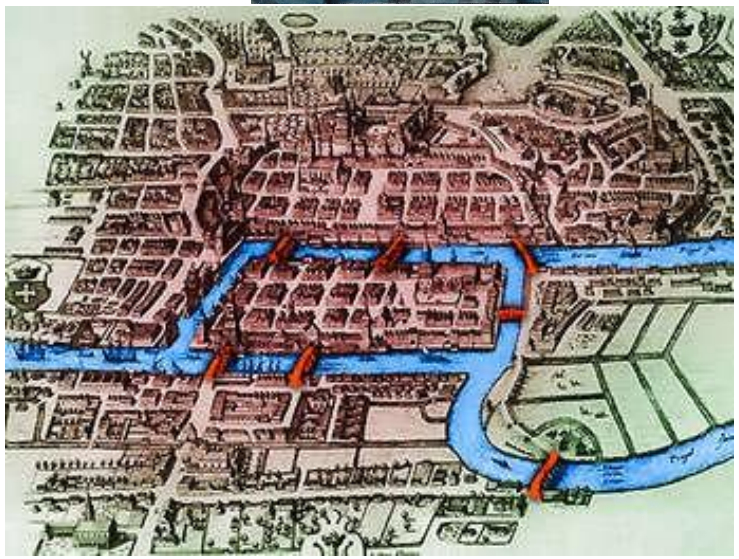


History: Königsberg bridges (1/2)

- The subject of graph theory began in the year 1736 when the great mathematician Leonhard Euler published a paper giving the solution to the following puzzle:

The town of Königsberg in Prussia (now Kaliningrad in Russia) was built at a point where two branches of the Pregel River came together. It consisted of an island and some land along the river banks.

These were connected by 7 bridges.



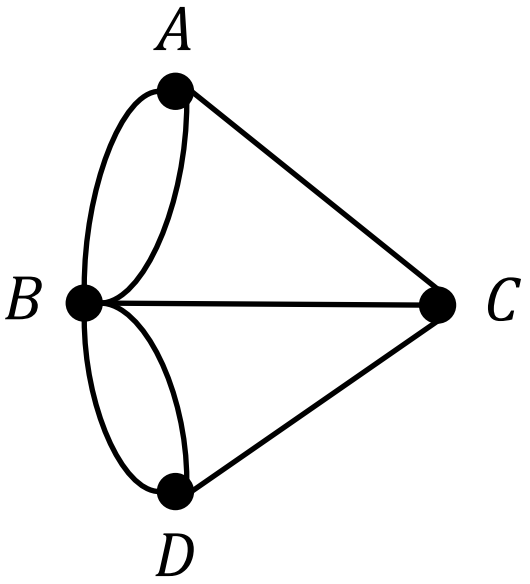
Euler, Leonhard. "Solutio problematis ad geometriam situs pertinentis." *Commentarii academiae scientiarum Petropolitanae* (1741): 128-140.

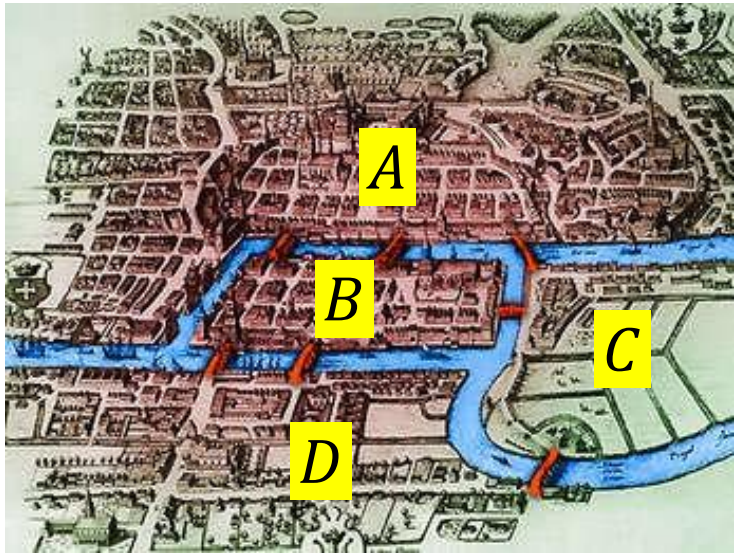
History: Königsberg bridges (2/2)

- Question: Is it possible to take a walk around town, starting and ending at the same location and crossing each of the 7 bridges exactly once?

In terms of this graph, the question is:

Is it possible to find a route through the graph that starts and ends at some vertex, one of *A*, *B*, *C*, or *D*, and traverses each edge exactly once?





Euler, Leonhard. "Solutio problematis ad geometriam situs pertinentis." *Commentarii academiae scientiarum Petropolitanae* (1741): 128-140.

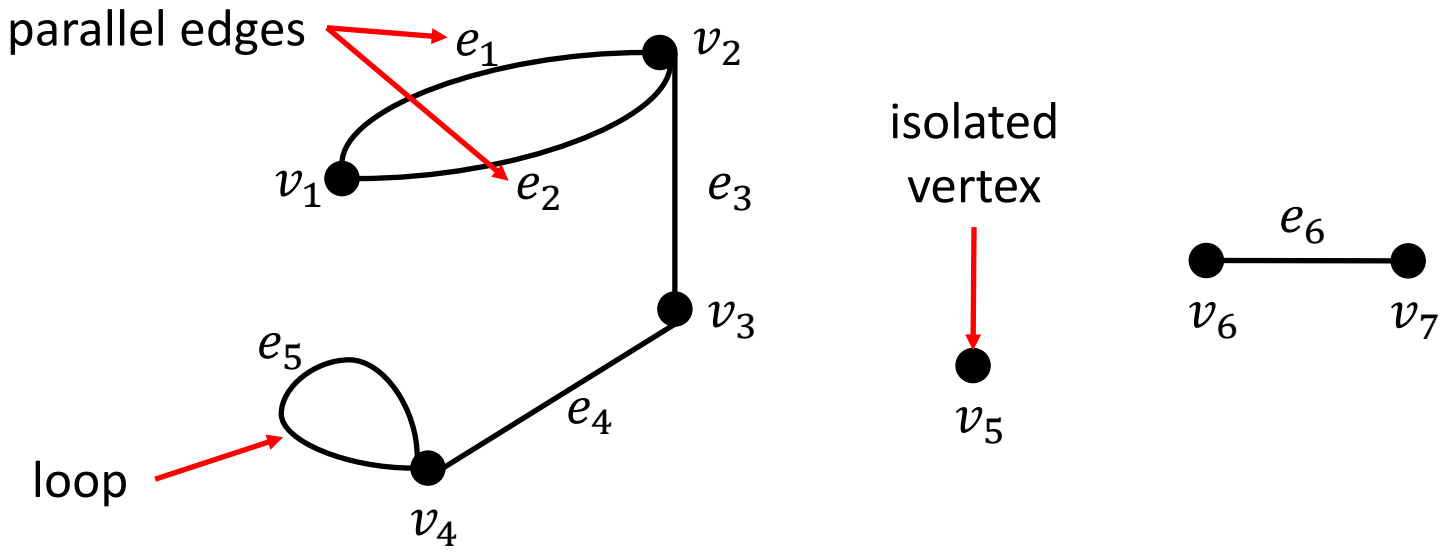
Outline

1. Introduction
2. Definition and basic properties
3. Trails, paths, and circuits
4. Graph representations
5. Isomorphisms
6. Graph traversals
   - Breadth-First Search (BFS)
   - Depth-First Search (DFS)

Definitions and basic properties

- A graph G consists of
  - a set of vertices $V(G)$, and
  - a set of edges $E(G)$.
- We often write $G = (V, E)$.

An edge connects one vertex to another, or a vertex to itself.



Graph definition

Definition 1.1. A graph G consists of 2 finite sets: a nonempty set V(G) of vertices and a set E(G) of edges, where each edge is associated with a set consisting of either one or two vertices called its endpoints.

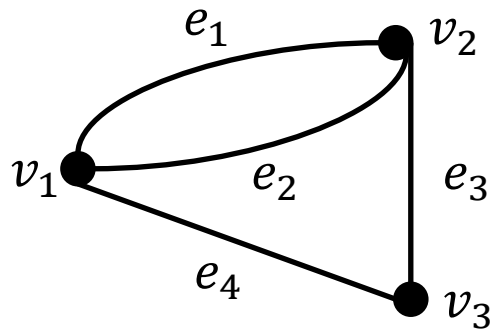
An edge is said to connect its endpoints; two vertices that are connected by an edge are called adjacent vertices; and a vertex that is an endpoint of a loop is said to be adjacent to itself.

An edge is said to be incident on each of its endpoints, and two edges incident on the same endpoint are called adjacent edges.

We write $e = (v, w)$ for an edge e incident on vertices v and w.

Example 1

- Consider the following graph:



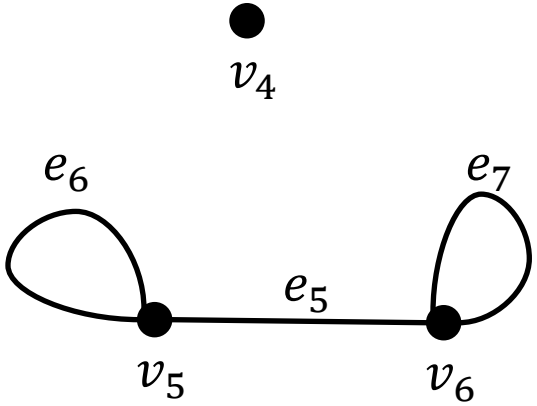
List the vertex set V, the edge set E, and specify each edge in E by naming its two incident vertices.

$$V = \{v_1, v_2, v_3, v_4, v_5, v_6\}$$

$$E = \{e_1, e_2, e_3, e_4, e_5, e_6, e_7\}$$

$e_1 = (v_1, v_2)$
$e_2 = (v_1, v_2)$
$e_3 = (v_2, v_3)$

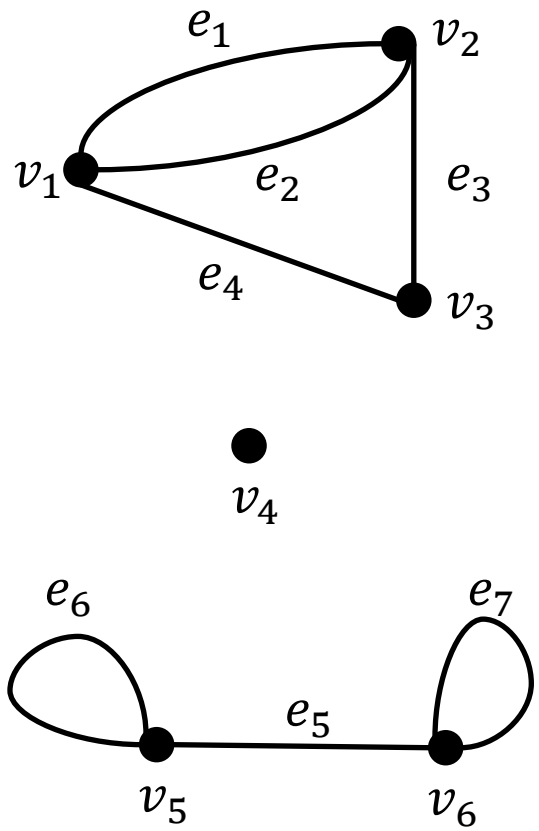
$e_4 = (v_1, v_3)$
$e_5 = (v_5, v_6)$
$e_6 = (v_5)$
$e_7 = (v_6)$



Example 2

- Consider the following graph:

Find all edges that are incident on v_1, all vertices that are adjacent to v_1, all edges that are adjacent to e_1, all loops, all parallel edges, all vertices that are adjacent to themselves, and all isolated vertices.



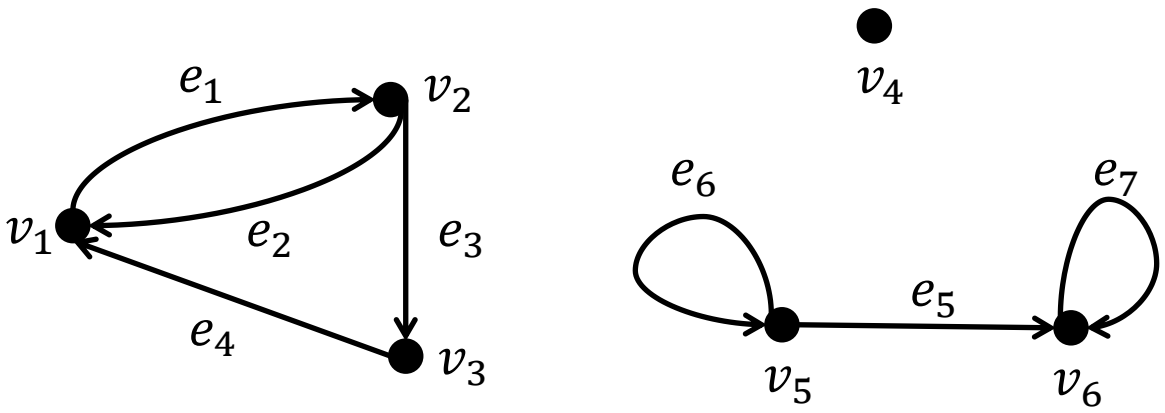
Edges incident on v_1: e_1, e_2 and e_4.
Vertices adjacent to v_1: v_2 and v_3.
Edges adjacent to e_1: e_2, e_3 and e_4.
Loops: e_6 and e_7.
e_1 and e_2 are parallel.

v_5 and v_6 are adjacent to themselves.
Isolated vertex: v_4.

Directed graphs

Definition 1.2. A directed graph, or digraph, G, consists of 2 finite sets: a nonempty set V(G) of vertices and a set D(G) of directed edges, where each edge is associated with an ordered pair of vertices called its endpoints.

If edge e is associated with the pair (v, w) of vertices, then e is said to be the (directed) edge from v to w. We write e = (v, w).



$$V = \{v_1, v_2, v_3, v_4, v_5, v_6\}$$

$$E = \{e_1, e_2, e_3, e_4, e_5, e_6, e_7\}$$

$e_1 = (v_1, v_2)$
$e_2 = (v_2, v_1)$
$e_3 = (v_2, v_3)$

$e_4 = (v_3, v_1)$
$e_5 = (v_5, v_6)$
$e_6 = (v_5)$
$e_7 = (v_6)$

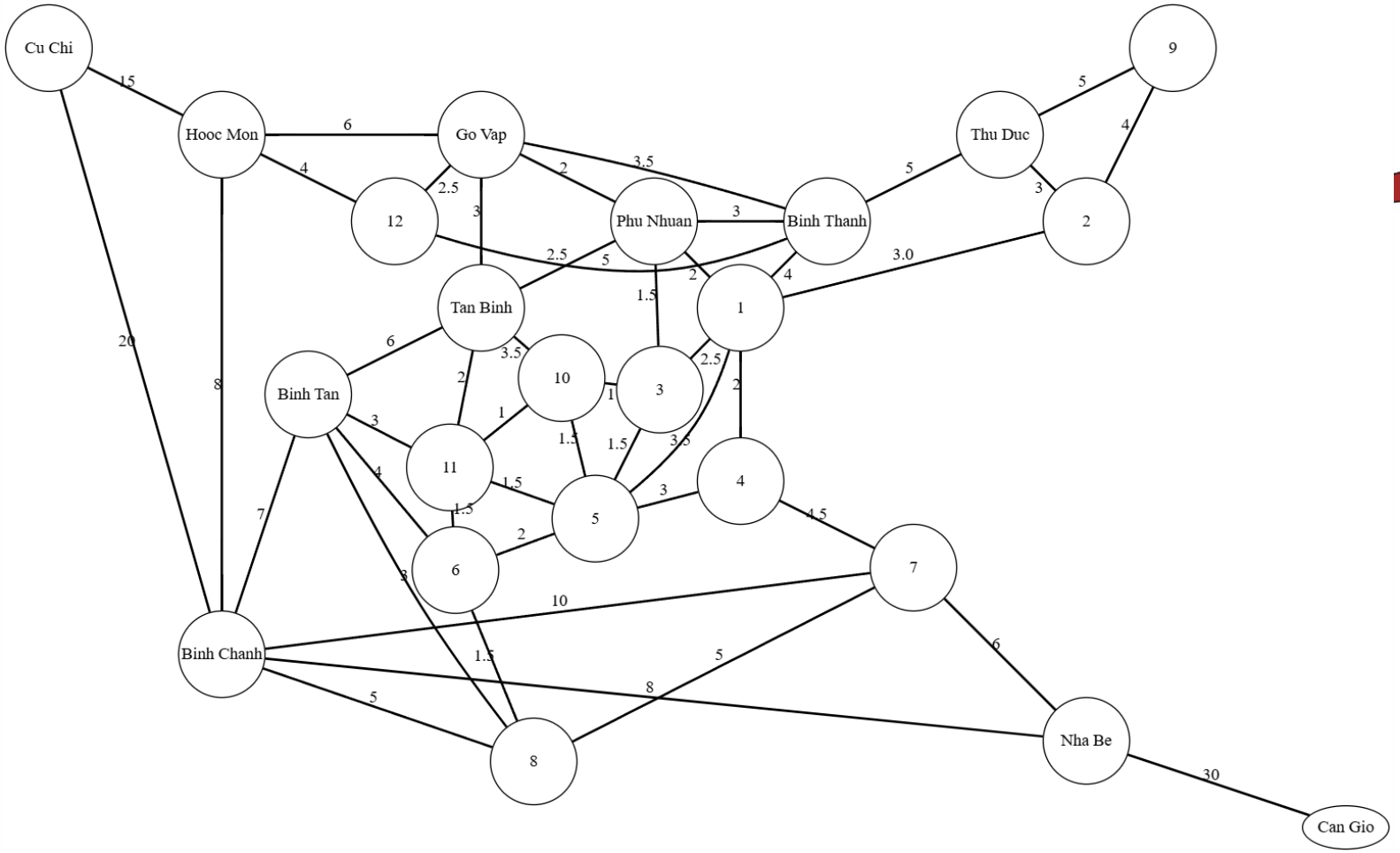
Modelling Graph Problems: map colouring problem (1/2)

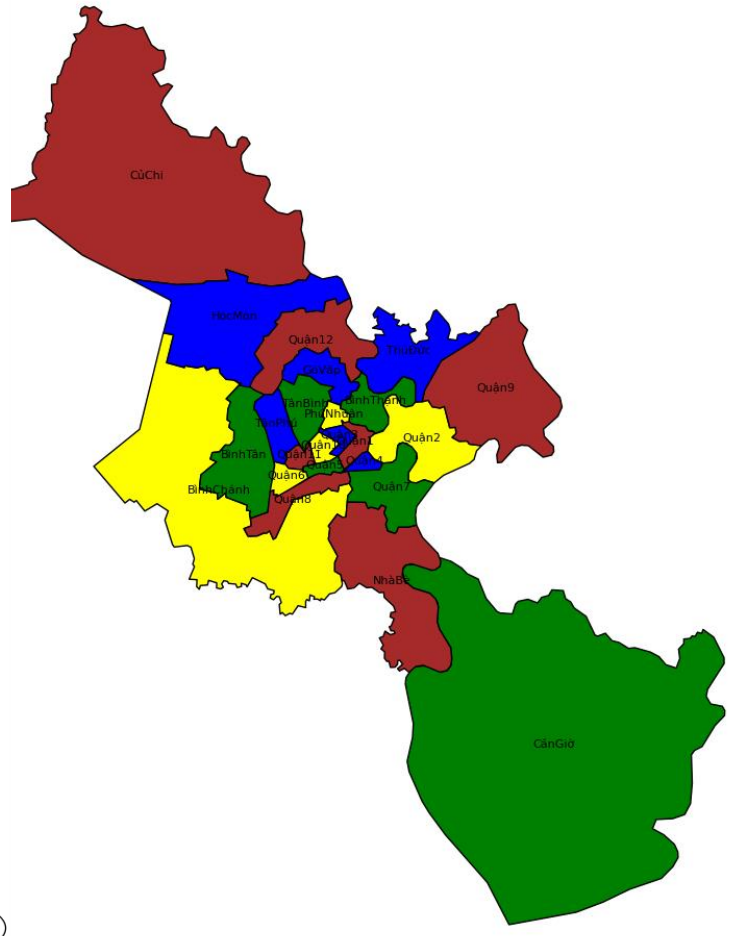
Map Colouring Problem

- Solve it as a graph problem.
- Draw a graph in which the vertices represent the districts with every edge joining two vertices represents the stat sharing a common border.
- Such two vertices cannot be coloured with the same colour.
- A vertex colouring of a graph is an assignment of colours to vertices so that no two adjacent vertices have the sam colour.

Modelling Graph Problems: map colouring problem (2/2)

Shall we add some colors to this map of Ho Chi Minh City?





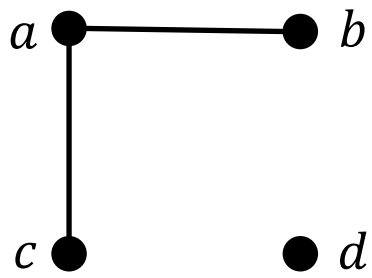
Other Vertex Colouring Problems

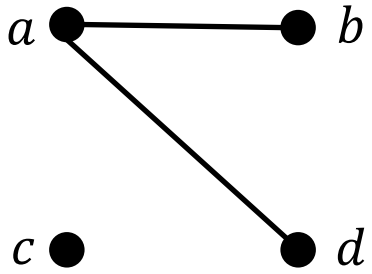
	If the vertices represent…	And two vertices are adjacent if ….	Then a vertex colouring can be used to…
1.	classes,	the corresponding classes have students in common,	schedule classes.
2.	radio stations,	the stations are close enough to interfere with each other,	assign non-interfering frequencies to the stations.
3.	traffic signals at an intersection,	the corresponding signals cannot be green at the same time,	designate sets of signals that can be green at the same time.

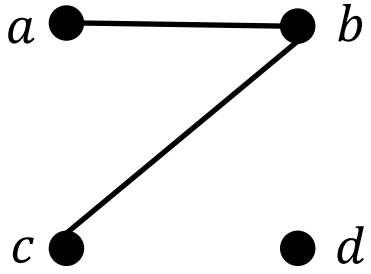
Simple graph

Definition 1. A simple graph is an undirected graph that does not have any loops or parallel edges.

- Example: Draw all simple graphs with the 4 vertices {a, b, c, d} and two edges, one of which is {a,b}.
- Each possible edge of a simple graph corresponds to a subset of two vertices. Hence there are $\binom{4}{2}$ = 6 such subsets. One is given.





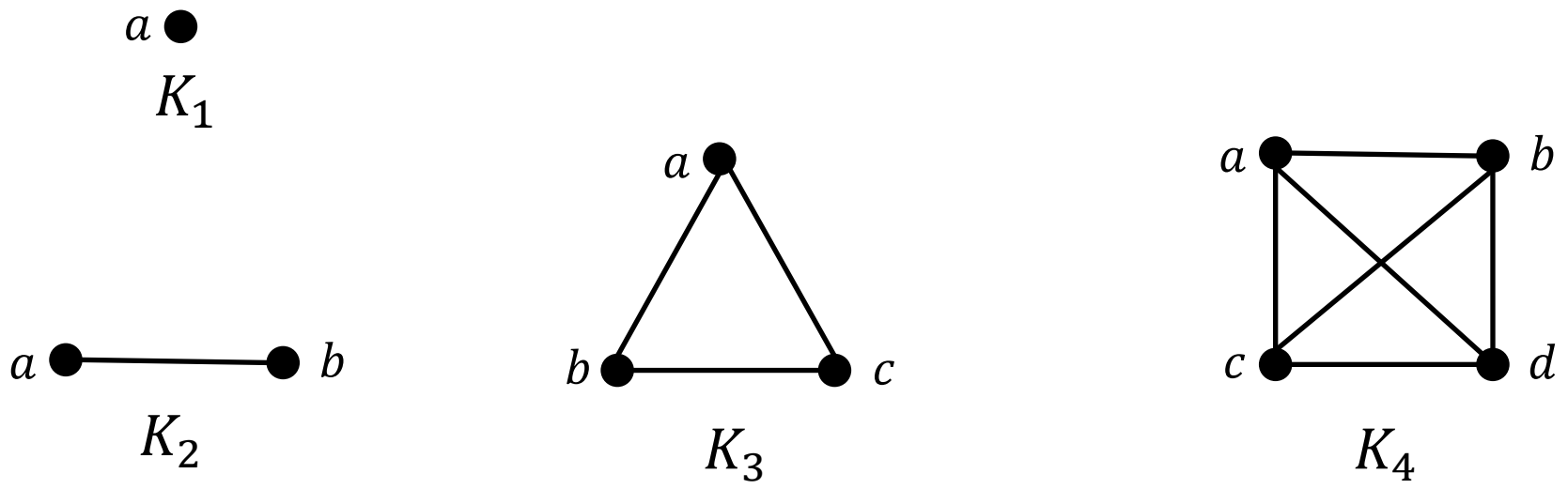


Draw the other two.

Complete Graph

Definition 1. A complete graph on n vertices, $n > 0$, denoted K_n, is a simple graph with n vertices and exactly one edge connecting each pair of distinct vertices

- Example: The complete graphs K_1, K_2, K_3 and K_4.



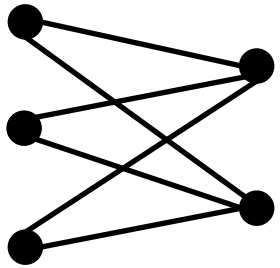
Complete Bipartite Graph

Definition 1.3. A complete bipartite graph on (m, n) vertices, where $m, n > 0$, denoted $K_{m,n}$, is a simple graph with distinct vertices $v_1, v_2, \dots, v_m$, and $w_1, w_2, \dots, w_n$ that satisfies the following properties:

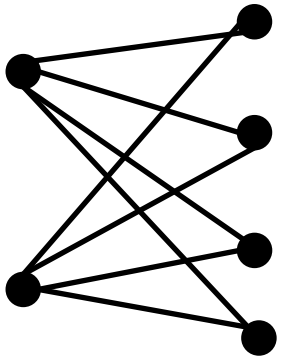
For all $i, k = 1, 2, \dots, m$ and for all $j, l = 1, 2, \dots, n$,

1. There is an edge from each vertex v_i to each vertex w_j.
2. There is no edge from any vertex v_i to any other vertex v_k.
3. There is no edge from any vertex w_j to any other vertex w_l.

$K_{3,2}$



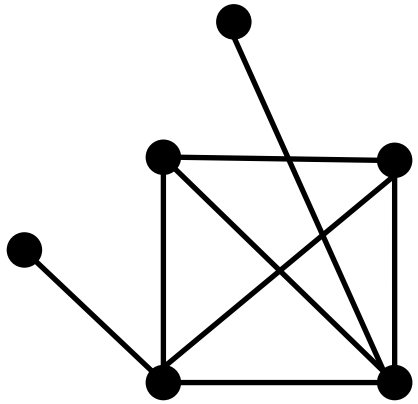
$K_{2,4}$

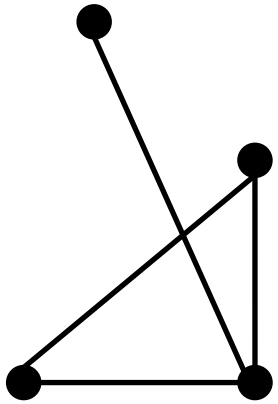


Subgraph of a Graph

Definition 1.4. A graph H is said to be a subgraph of graph G if, and only if, every vertex in H is also a vertex in G, every edge in H is also an edge in G, and every edge in H has the same endpoints as it has in G.

A graph *G*





A subgraph H of *G*

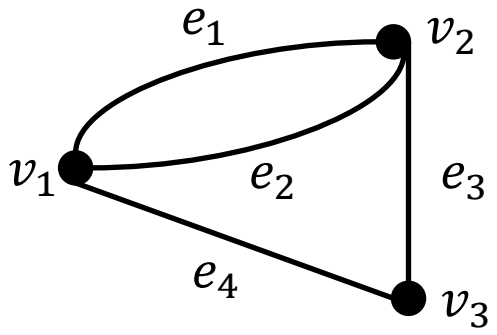
Degree of a Vertex and Total Degree of a Graph

Definition 1.5. Let G be a graph and v be a vertex of G. The degree of v, denoted deg(v), equals the number of edges that are incident on v, with an edge that is a loop counted twice.

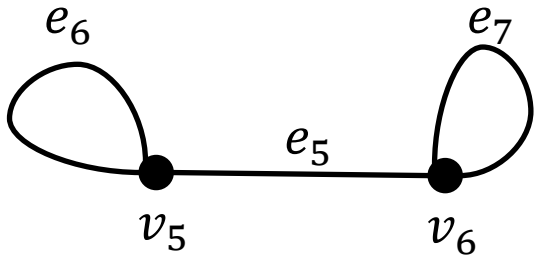
The total degree of G is the sum of the degrees of all the vertices of G.

The degree of a vertex can be obtained from the drawing of a graph by counting how many end segments of edges are incident on the vertex.

$\deg(v_1) = 3$

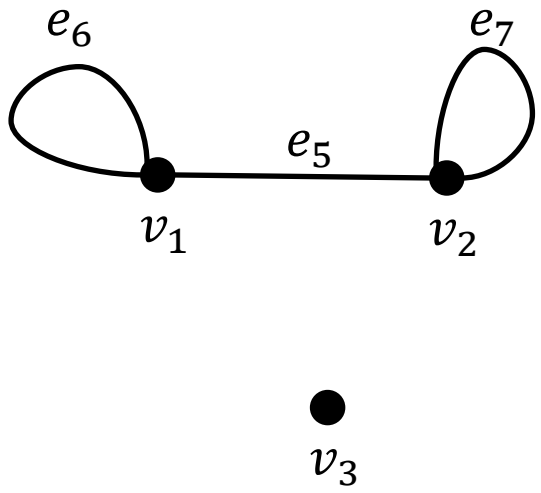


$\deg(v_5) = 3$



Example 1

- Find the degree of each vertex of the graph *G* shown behind. Then find the total degree of *G*.



$\deg(v_3) = 0$
$\deg(v_1) = 3$
$\deg(v_2) = 3$
Total degree of $G = 6$

The concept of degree

Theorem 1.1 (The Handshake Theorem). If G is any graph, then the sum of the degrees of all the vertices of G equals twice the number of edges of G. Specifically, if the vertices of G are $v_1, v_2, \dots, v_n$, where $n \geq 0$, then

$$The \text{ total degree of G} = \deg(v_1) + \deg(v_2) + \cdots + \deg(v_n)$$

$$= 2(\text{the number of edges of } G).$$

Corollary 1.1. The total degree of a graph is even.

Corollary 1.2. In any graph there are an even number of vertices of odd degree.

Outline

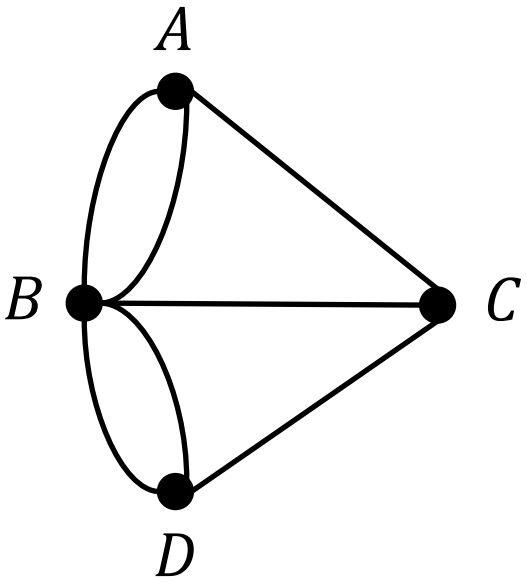
1. Introduction
2. Definition and basic properties
3. Trails, paths, and circuits
4. Graph representations
5. Isomorphisms
6. Graph traversals
   - Breadth-First Search (BFS)
   - Depth-First Search (DFS)

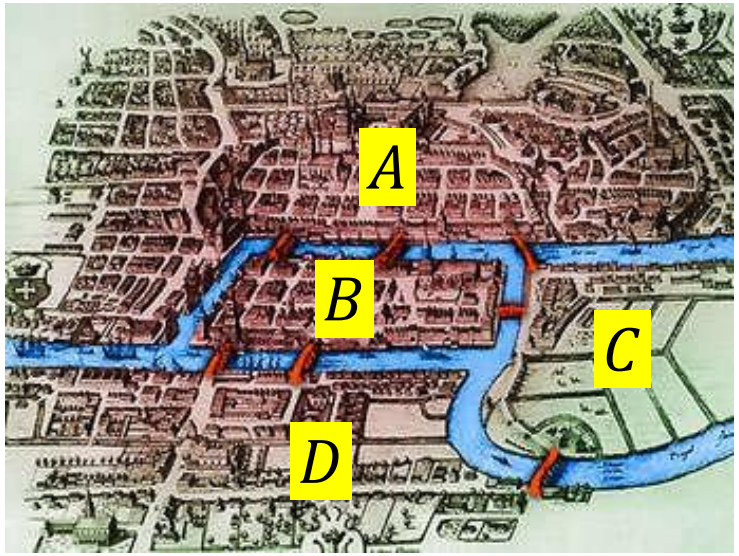
History: Königsberg bridges (Euler circuit)

- Question: Is it possible to take a walk around town, starting and ending at the same location and crossing each of the 7 bridges exactly once?

In terms of this graph, the question is:

Is it possible to find a route through the graph that starts and ends at some vertex, one of *A*, *B*, *C*, or *D*, and traverses each edge exactly once?

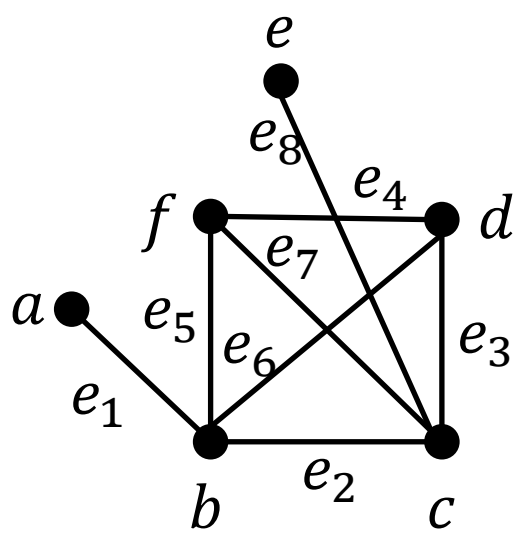




Euler, Leonhard. "Solutio problematis ad geometriam situs pertinentis." *Commentarii academiae scientiarum Petropolitanae* (1741): 128-140.

Traversal in graph

Travel in a graph is accomplished by moving from one vertex to another along a sequence of adjacent edges. In the graph below, for instance, you can go from a to d by taking e_1 to b and then e_6 to d. This is represented by writing ae_1be_6d.



Or, you could take a roundabout route:

$$ae_1be_5fe_7ce_8ee_8ce_2be_6d$$

Walk, Trail, Path

Definition 1.6. Let G be a graph, and let v and w be vertices of G.

A walk from v to w is a finite alternating sequence of adjacent vertices and edges of G. Thus a walk has the form

$$v_0 e_1 v_1 e_2 \dots v_{n-1} e_n v_n,$$

where the v's represent vertices, the e's represent edges, $v_0 = v, v_n = w$, and for all $i \in \{1, 2, \dots, n\}$, v_{i-1} and v_i are the endpoints of e_i.

The trivial walk from v to v consist of the single vertex v.

A trail from v to w is a walk from v to w that does not contain a repeated edge.

A path from v to w is a trail that does not contain a repeated vertex.

Closed Walk, Circuit, Simple Circuit

Definition 1.7. A closed walk is a walk that starts and ends at the same vertex.

A circuit (or cycle) is a closed walk that contains at least one edge and does not contain a repeated edge.

A simple circuit (or simple cycle) is a circuit that does not have any other repeated vertex except the first and last.

Quiz

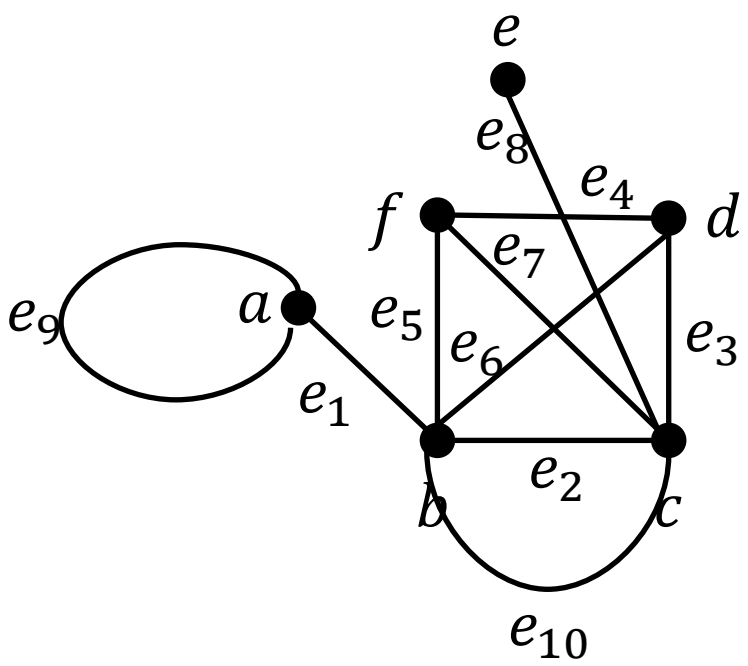
In this graph, determine which of the following walks are trails or paths:

a) $ae_1be_2ce_3de_4f$.

b) $ae_1be_2ce_2be_6de_4f$.

c) $be_{10}ce_3de_4fe_7ce_2b$.

d) ae_9a.



a) Path/Trail.
b) Neither trail nor path.
c) Trail; not a path.
d) Trail; not a path.

Walk, Trail, Path, Closed Walk, Circuit, Simple Circuit

	Repeated edge?	Repeated vertex?	Starts and Ends at the same point?	Must contain at least one edge?
Walk	ALLOWED	ALLOWED	ALLOWED	NO
Trail	NO	ALLOWED	ALLOWED	NO
Path	NO	NO	NO	NO
Closed walk	ALLOWED	ALLOWED	YES	NO
Circuit (Cycle)	NO	ALLOWED	YES	YES
Simple circuit	NO	First and last only	YES	YES

Quiz

In this graph, determine which of the following walks are trails, paths, circuits, or simple circuits.

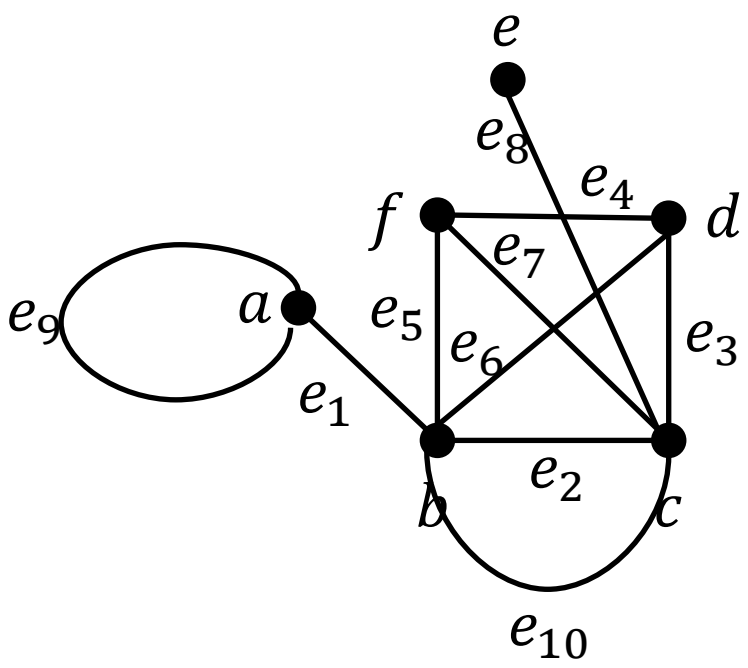
a) $ae_1be_2ce_3de_4f$.

b) $ae_1be_2ce_2be_6de_4f$.

c) $be_{10}ce_3de_4fe_7ce_2b$.

d) ae_9a.

e) $be_{10}ce_3de_4fe_5b$.



a) Path/Trail.
b) Neither trail nor path.
c) Circuit; not a simple circuit.

d) Simple circuit
e) Simple circuit

Cautions

- Because many of the major developments in graph theory have occurred relatively recently (300 years of graph theory and 70 years for major development) and across various disciplines, the terminology used in the field is not yet fully standardized.
- For instance, what this book refers to as *a graph* is sometimes called a *multigraph* in other sources. Similarly, a *simple graph* may be referred to simply as a *graph*. The term *vertex* is often replaced with *node*, and *edge* may also be called an *arc*.
- Likewise, some sources use *path* in place of *trail*, *simple path* instead of *path*, and *cycle* instead of *simple circuit*.
- The terminology used in this slide aligns with some of the most widely accepted conventions. However, when consulting other resources, it's important to verify their definitions, as they may differ.

Connectedness

Definition 1.8. **Two vertices** *v* and *w* of a graph *G* are **connected** if, and only if, there is a walk from *v* to *w*.

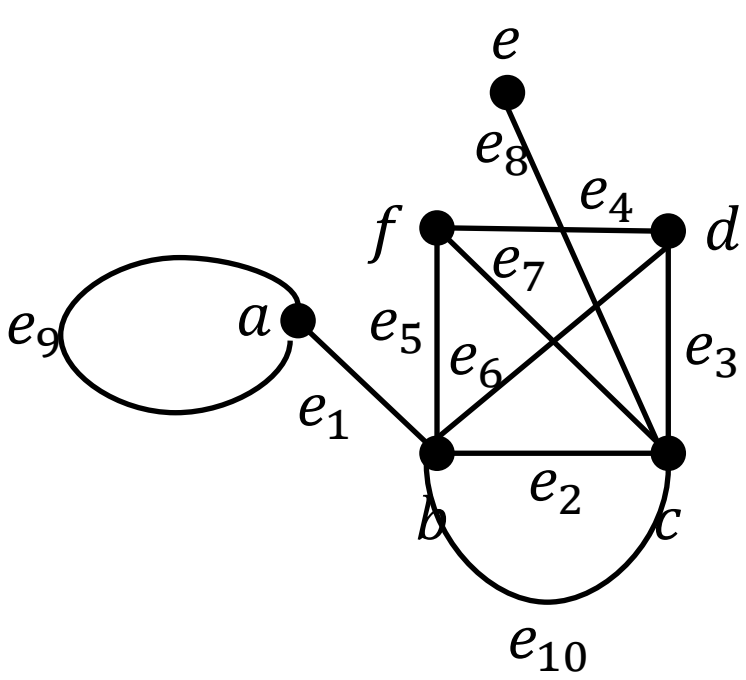
The graph *G* is connected if, and only if, given *any* two vertices *v* and *w* in *G*, there is a walk from *v* to *w*. Symbolically,

G is connected iff ∀vertices $v, w \in V(G)$, ∃ a walk from *v* to *w*.

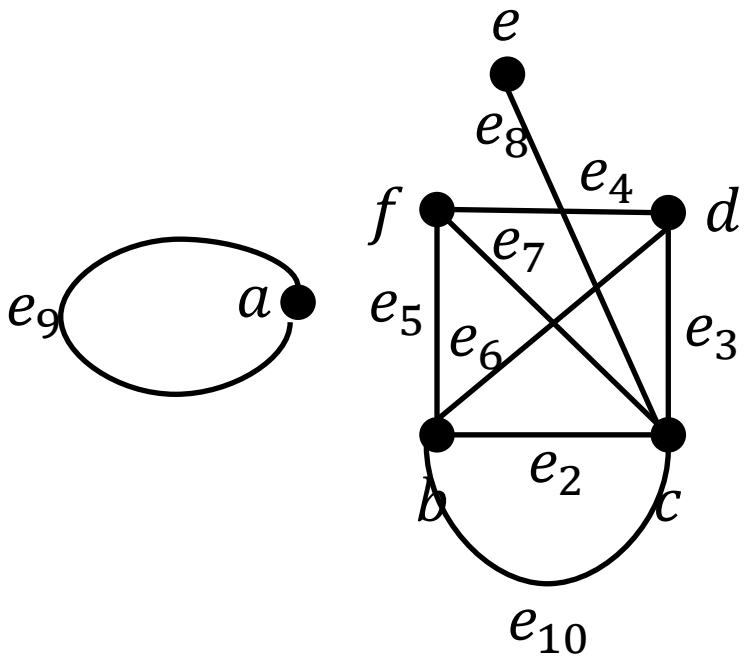
- A graph is connected if it is possible to travel from any vertex to any other vertex along a sequence of adjacent edges of the graph.

Example

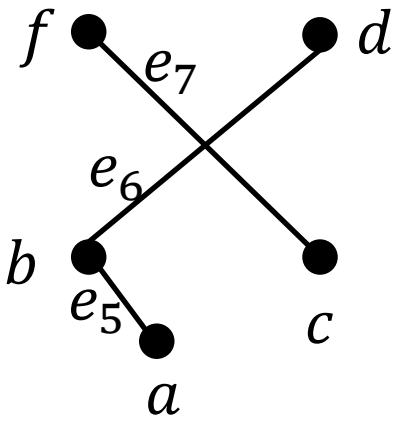
- Which of the following graphs are connected?



(a)



(b)



(c)

Useful facts

Lemma 1.1. Let G be a graph.

If G is connected, then any two distinct vertices of G can be connected by a path.

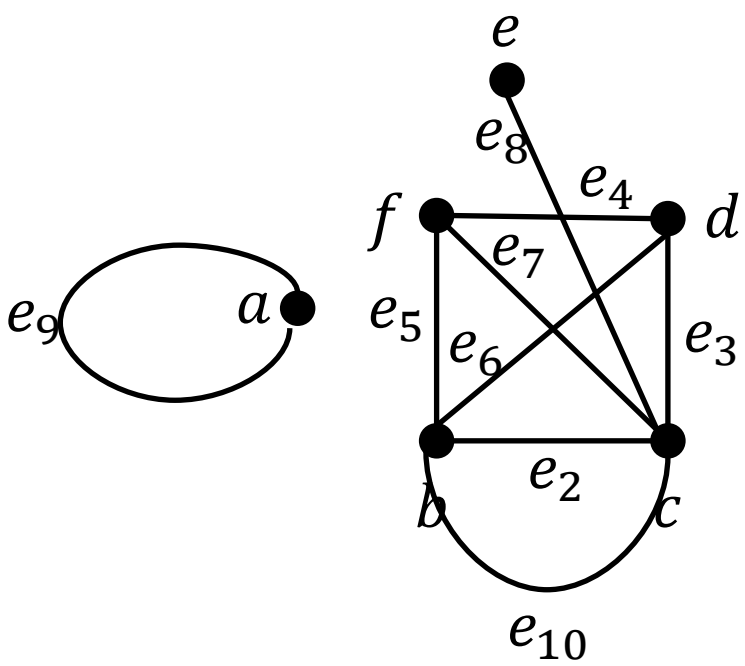
If vertices v and w are part of a circuit in G and one edge is removed from the circuit, then there still exists a trail from v to w in G.

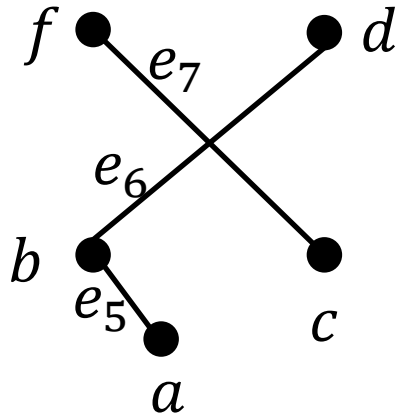
If G is connected and G contains a circuit, then an edge of the circuit can be removed without disconnecting G.

- Some useful facts relating circuits and connectedness are collected in the above lemma.

Connected Component

- The graphs in (b) and (c) are both made up of three pieces, each of which is itself a connected graph.
- A connected component of a graph is a connected subgraph of largest possible size.





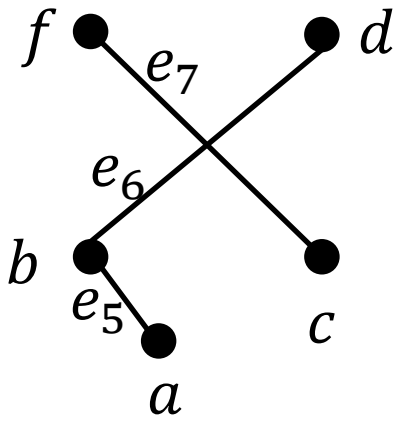
Connected Component

Definition 1.9. A graph H is a connected component of a graph G if, and only if,

1. The graph H is a subgraph of G;
2. The graph H is connected; and
3. No connected subgraph of G has H as a subgraph and contains vertices or edges that are not in H.

Example

- Find all connected components of the following graph G.



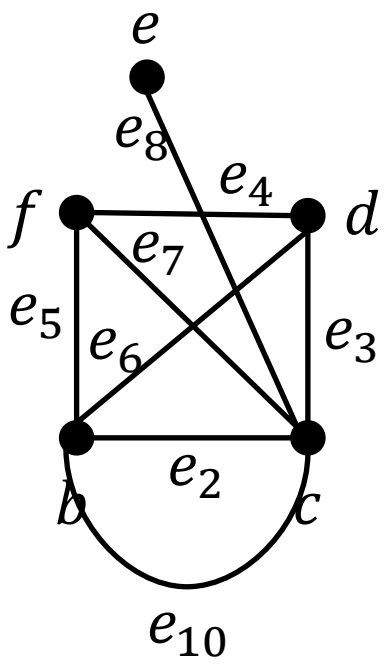
G has 2 connected components H_1 and H_2 with vertex sets V_1 and V_2 and edge sets E_1 and E_2 where

$$V_1 = \{a, b, d\}, E_1 = \{e_5, e_6\}$$
$$V_2 = \{f, c\}, E_2 = e_7$$

Vertex cut and Bridges

Definition 1.10. An edge e in graph $G = (V, E)$ is a bridge if removing it from G causes a new graph $G' = (V, E \setminus \{e\})$ disconnected.

A vertex v in graph $G = (V, E)$ is a vertex cut if removing it from G causes a new graph $G' = (V \setminus \{v\}, E)$ disconnected.



e_8 is a bridge.

e_{10} is NOT a bridge.

c is a vertex cut.

Euler Circuits

Definition 1.11. Let G be a graph. An Euler circuit for G is a circuit that contains every vertex and every edge of G.

That is, an Euler circuit for G is a sequence of adjacent vertices and edges in G that has at least one edge, starts and ends at the same vertex, uses every vertex of G at least once, and uses every edge of G exactly once.

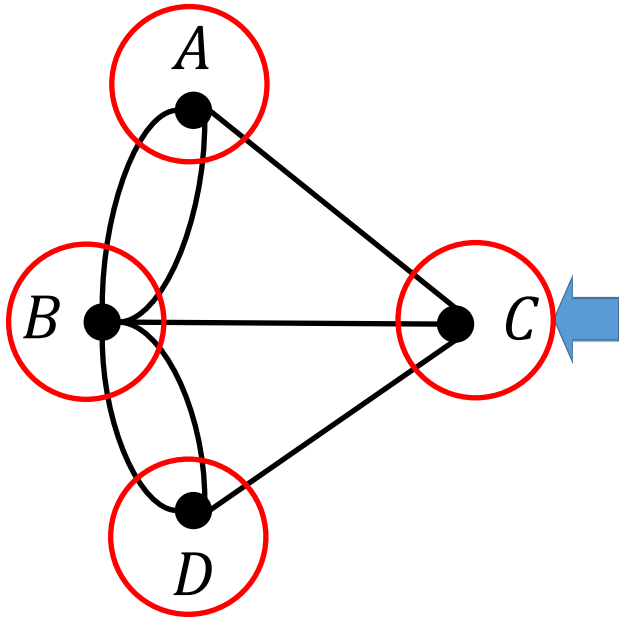
Theorem 1.2. If a graph has an Euler circuit, then every vertex of the graph has a positive even degree. In other words, if some vertex of a graph has odd degree, then the graph does not have an Euler circuit.

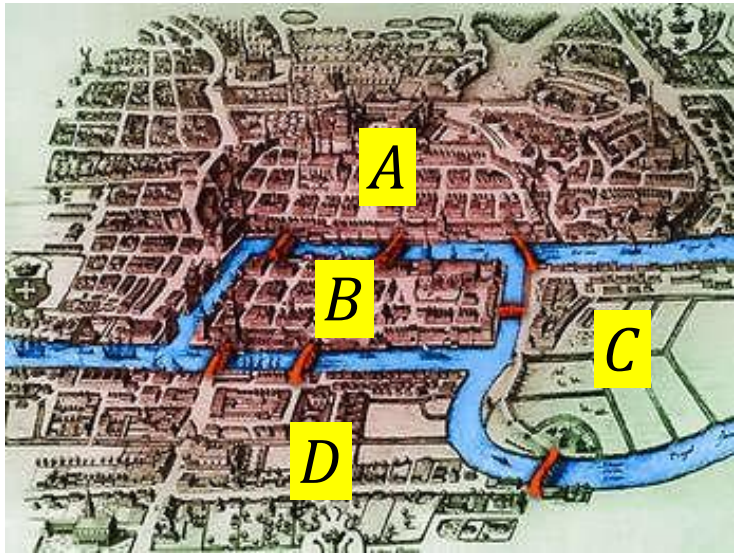
History: Königsberg bridges (Euler circuit)

- Question: Is it possible to take a walk around town, starting and ending at the same location and crossing each of the 7 bridges exactly once?

A vertex's degree is odd

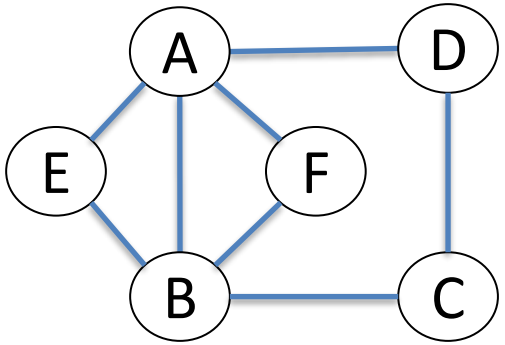
➔ NO such walk exists



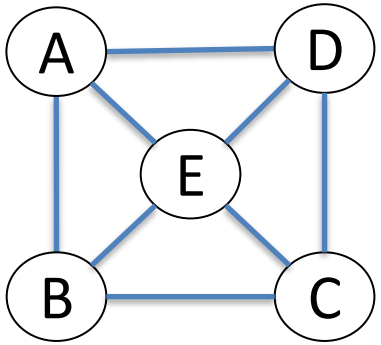


Euler, Leonhard. "Solutio problematis ad geometriam situs pertinentis." *Commentarii academiae scientiarum Petropolitanae* (1741): 128-140.

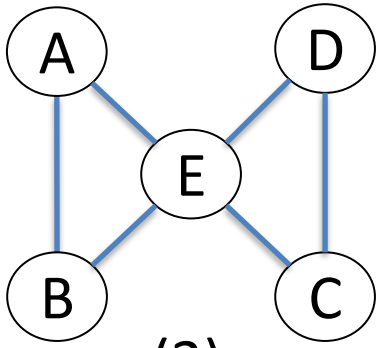
- Does each of the following graphs have an Euler circuit?



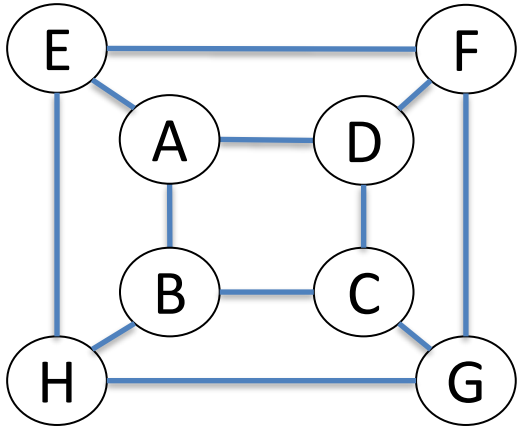
(1)



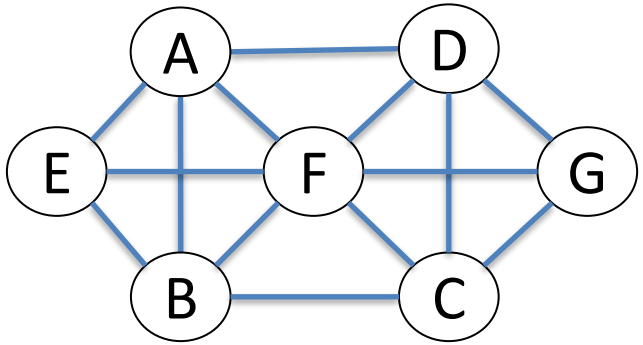
(2)



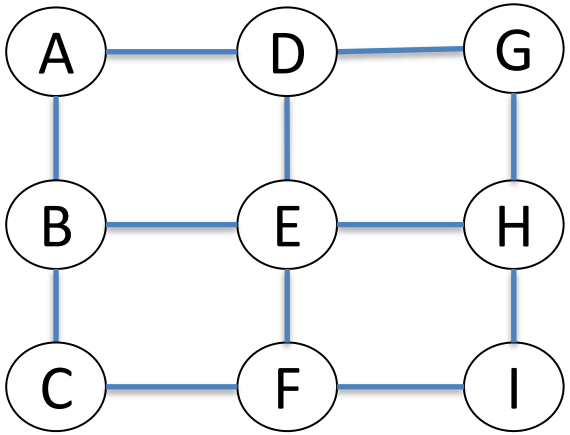
(3)



(4)



(5)



(6)

Euler Circuits

Is the converse of Theorem 1.2 true?

If every vertex of a graph has an even degree, then the graph has an Euler circuit.

Theorem 1.2. If a graph has an Euler circuit, then every vertex of the graph has a positive even degree. In other words, if some vertex of a graph has odd degree, then the graph does not have an Euler circuit.

Useful facts

Theorem 1.3. If a graph G is connected and the degree of every vertex of G is a positive even integer, then G has an Euler circuit.

- Combine Theorems 1.2 and 1.3, we get the sufficient and necessary condition to have an Euler circuit in a grap.

Theorem 1.4. A graph G has an Euler circuit if, and only if, G is connected and every vertex of G has positive even degree.

- A corollary to Theorem 1.4 gives a criterion for determining when it is possible to find a walk from one vertex of a graph to another, passing through every vertex of the graph at least once and every edge of the graph exactly once.

Conditions to have an Euler circuit

Theorem 1.4. A graph G has an Euler circuit if, and only if, G is connected and every vertex of G has positive even degree.

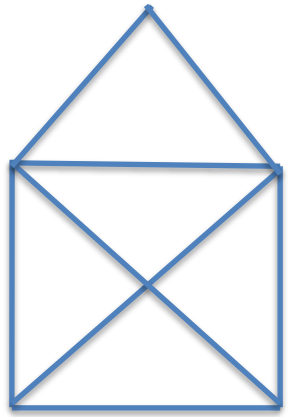
Euler Trail

Definition 1.12. Let G be a graph, and let v and w be two distinct vertices of G. An Euler trail/path from v to w is a sequence of adjacent edges and vertices that starts at v, ends at w, passes through every vertex of G at least once, and traverses every edge of G exactly once.

Corollary 1.3. Let G be a graph, and let v and w be two distinct vertices of G. There is an Euler trail from v to w if, and only if, G is connected, v and w have odd degree, and all other vertices of G have positive even degree.

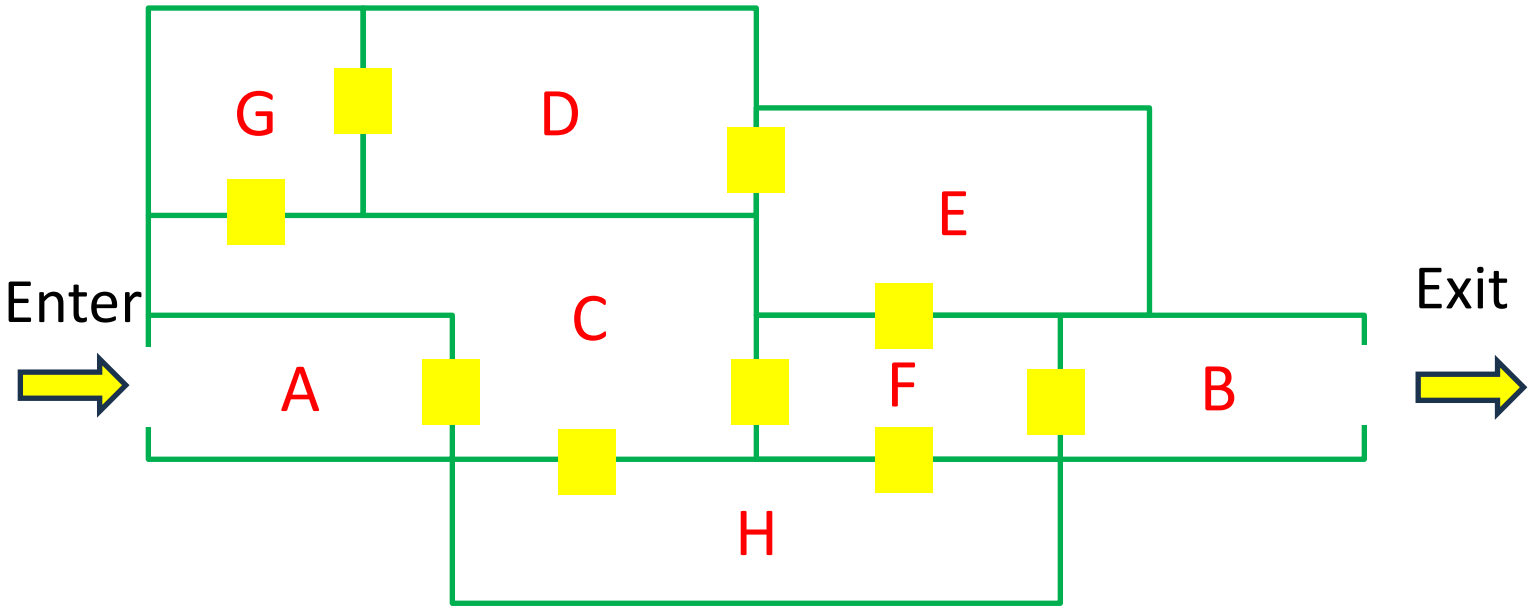
Exercise 1

- The following graphs do not have an Euler circuit. Do they have an Euler trail/path?



Exercise 2

- The diagram below shows the floor plan of a house that is open for public viewing. Is it possible to find a path that starts in Room A and ends in Room B, passing through every interior doorway of the house exactly once? If such a path exists, determine it.



Hierholzer's Algorithm for Identifying Eulerian Circuits (words)

Given: An Eulerian graph G

1. Finding an initial circuit.
2. Looking for any vertex on the current circuit that has unused edges.
3. From that vertex, find a new circuit using only unused edges and return to that vertex.
4. Splice the new circuit into the old one.
5. Repeat until all edges are used.

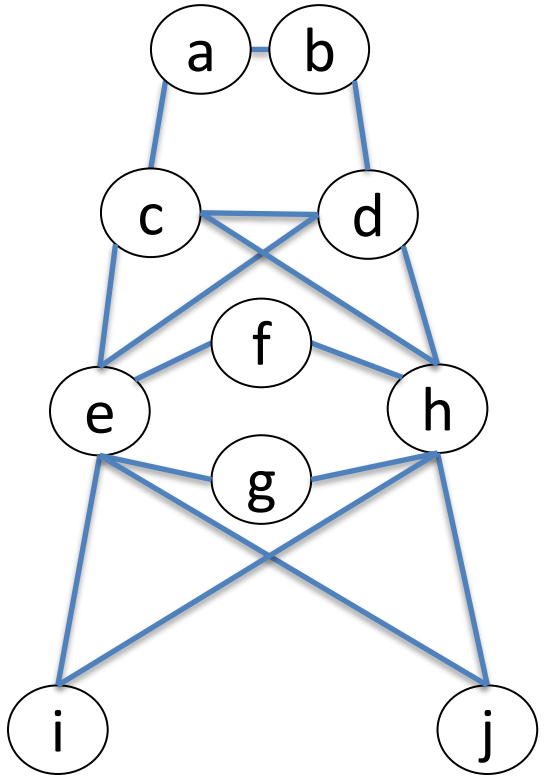
Hierholzer's Algorithm for Identifying Eulerian Circuits (details)

Given: An Eulerian graph G

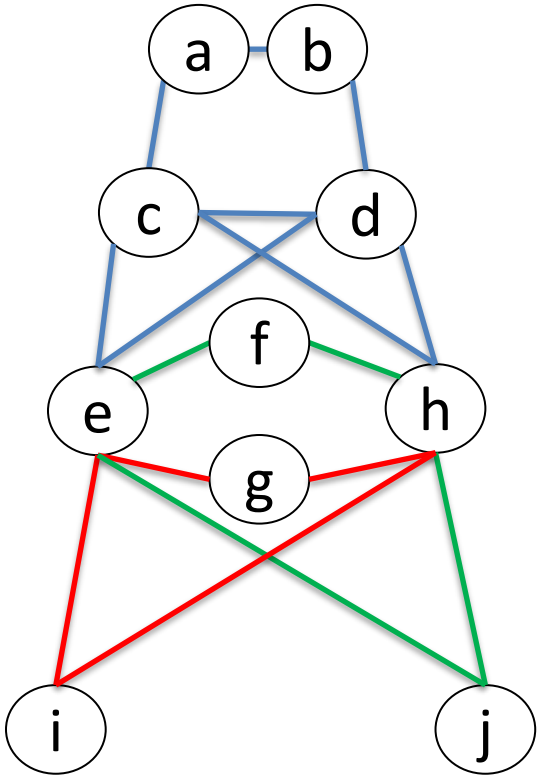
1. Identify a circuit in G and call it R_1. Mark the edges of R_1. Let $i = 1$.
2. If R_i contains all edges of G, then stop (since R_i is an Eulerian circuit).
3. If R_i does not contain all edges of G, then let v_i be a vertex on R_i that is incident with an unmarked edge e_i.
4. Build a circuit Q_i, starting at vertex v_i and using edge e_i. Mark the edges of Q_i.
5. Create a new circuit R_{i+1} by patching the circuit Q_i into R_i at v_i.
6. Increment i by 1, and go to Step 2.

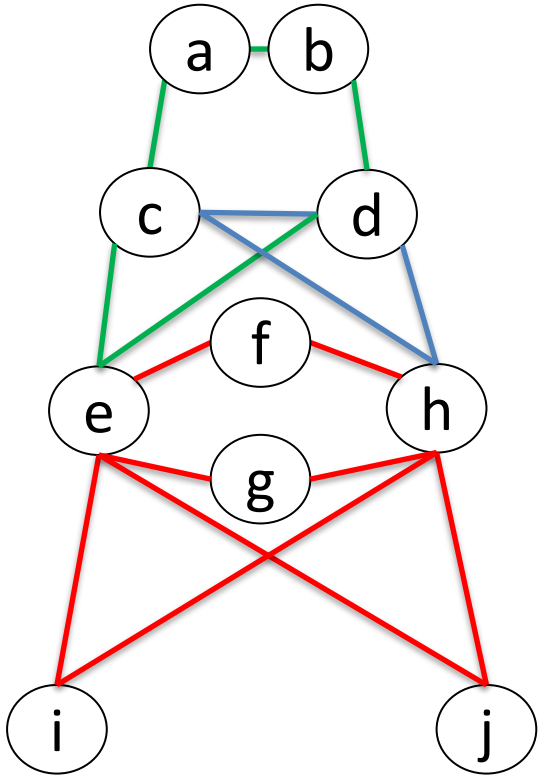
Example

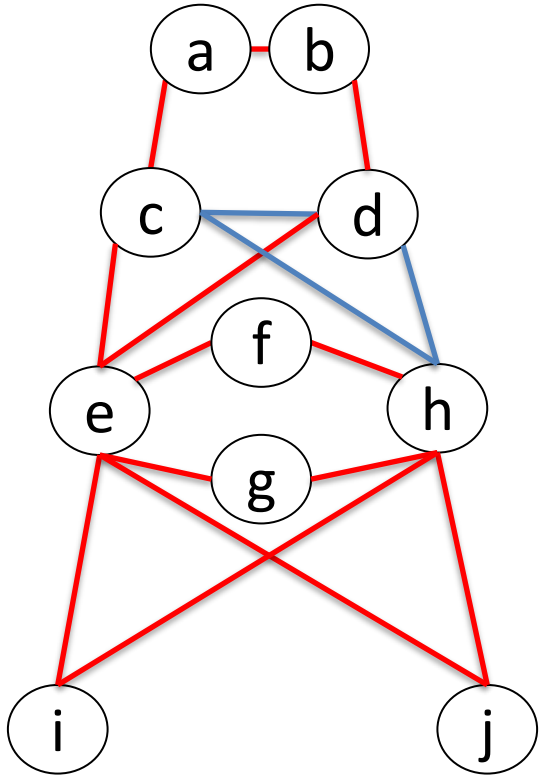
- Use Hierholzer's algorithm to find an Eulerian circuit in the following graph by using $R_1: \{e, g, h, i, e\}$ as your initial circuit.

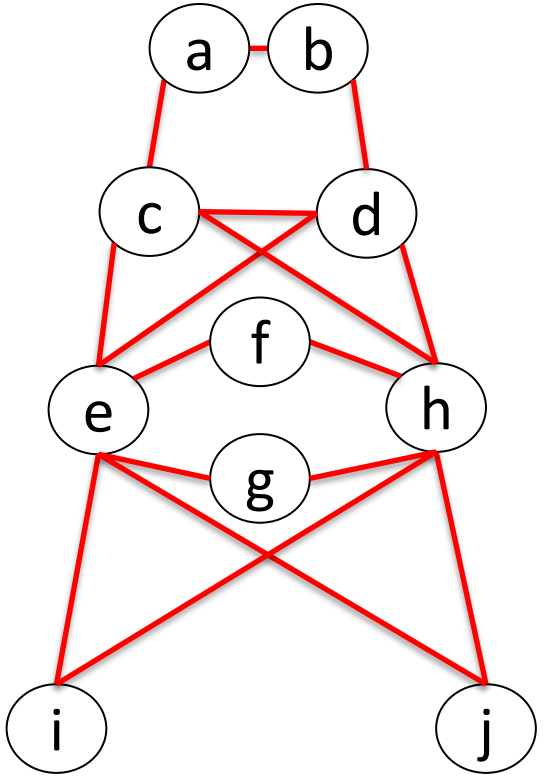


Solution









$R_1 = \{e, g, h, i, e\}$ $R_2 = \{e, f, h, j, e, g, h, i, e\}$

$Q_1 = \{e, f, h, j, e\}$ $Q_2 = \{e, c, a, b, d, e\}$

$R_3 = \{e, c, a, b, d, e, f, h, j, e, g, h, i, e\}$

$Q_3 = \{c, d, h, c\}$

$R_3 = \{e, c, d, h, c, a, b, d, e, f, h, j, e, g, h, i, e\}$

STOP

Hierholzer's Algorithm: Complexity

Given: An Eulerian graph G

1. Identify a circuit in G and call it R_1. Mark the edges of R_1. Let $i = 1$.
2. If R_i contains all edges of G, then stop (since R_i is an Eulerian circuit).
3. If R_i does not contain all edges of G, then let v_i be a vertex on R_i that is incident with an unmarked edge e_i.
4. Build a circuit Q_i, starting at vertex v_i and using edge e_i. Mark the edges of Q_i.
5. Create a new circuit R_{i+1} by patching the circuit Q_i into R_i at v_i.
6. Increment i by 1, and go to Step 2.

Total: $O(|V| + |E|)$

$O(|E|)$ by BFS or DFS

$O(1)$ by keeping a counter

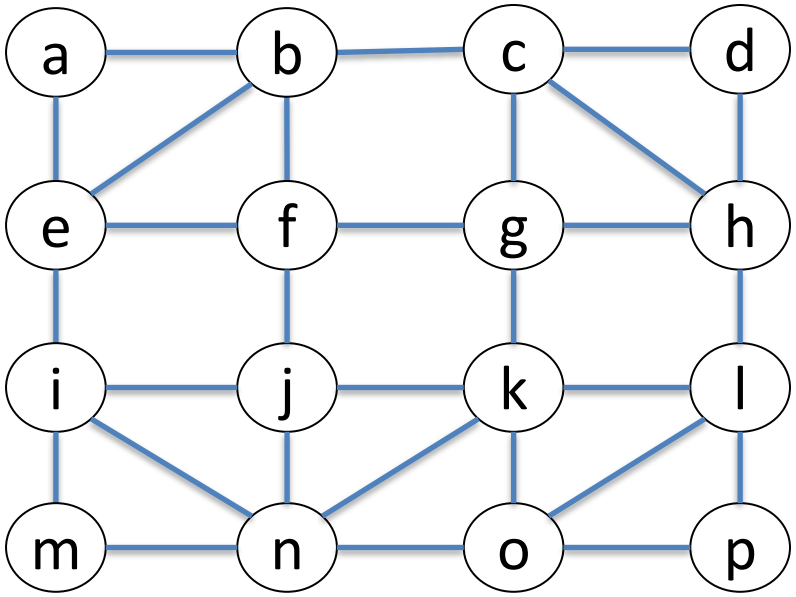
$O(|V|)$

$O(|E|)$ by BFS or DFS

$O(1)$ by using linked lists

Quiz

- Use Hierholzer's algorithm to find an Eulerian circuit in the following graph by using $R_1: a, b, c, g, f, j, i, e, a$ as your initial circuit.



- Degrees of n and k are odd ➔ No Eulerian circuit.
- Only 2 nodes are odd ➔ There exists Eulerian trail.

Fleury's Algorithm for Identifying Eulerian Circuits

Given: An Eulerian graph G, with all of its edges unmarked.

1. Choose a vertex v and call it the "lead vertex."
2. If all edges of G have been marked, then stop. Otherwise, continue to **Step 3**.
3. Among all edges incident on the lead vertex, choose, if possible, one that is **not a bridge** of the subgraph formed by the unmarked edges.

   If this is not possible, choose any edge incident on the lead. Mark this edge and let its other end vertex be the new lead vertex.
4. Go to **Step 2**.

Fleury's Algorithm: complexity

Given: An Eulerian graph G, with all of its edges unmarked.

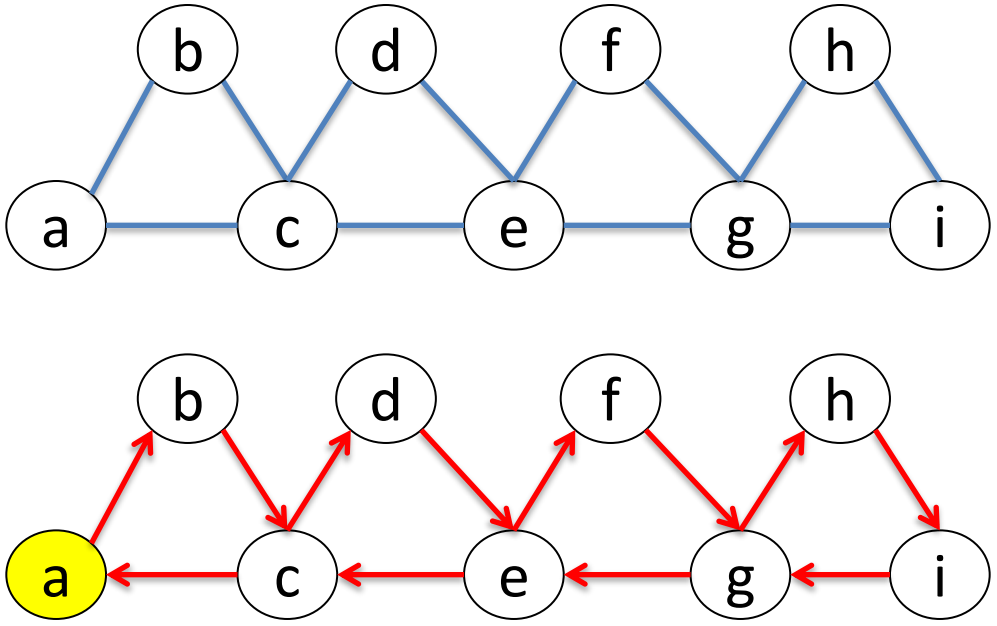
1. Choose a vertex v and call it the "lead vertex."
2. If all edges of G have been marked, then stop. Otherwise, continue to **Step 3**.
3. Among all edges incident on the lead vertex, choose, if possible, one that is **not a bridge** of the subgraph formed by the unmarked edges. If this is not possible, choose any edge incident on the lead. Mark this edge and let its other end vertex be the new lead vertex.
4. Go to **Step 2**.

Comparison table between Hierholzer's Algorithm and Fleury's Algorithm

Feature	Hierholzer's Algorithm	Fleury's Algorithm
Approach Type	Circuit growing via patching smaller circuits	Greedy traversal, avoiding bridges if possible
Starting Point	Any vertex	Any vertex
Edge Selection Rule	Arbitrarily choose any unmarked edge to form a circuit	Prefer non-bridge edges; use bridge only when necessary
Bridge Checking	Not needed	Required at each step to avoid disconnecting the graph
Efficiency (Time Complexity)	**O($\|V\| + \|E\|$)** (linear in number of edges)	Up to $O(\|E\|^2)$ due to repeated bridge checking
Implementation Complexity	More complex due to recursive patching or merging	Conceptually simple but computationally expensive
Practical Use	Preferred in implementations due to efficiency	Educational or for small graphs; not suitable for large graphs due to inefficiency
Edge Marking	Marks edges during circuit formation and merging	Marks each edge once it is traversed
Graph Modification	Builds multiple circuits and merges them	Modifies the graph by removing edges after traversal
Commonly Taught In	Algorithm design or computer science courses	Introductory graph theory courses (for intuitive understanding)

Quiz

- Use Fleury's algorithm to find an Eulerian circuit in the following graph by using vertex a as your initial circuit.



Hamiltonian Circuits (1/

- Recall a theorem related with Eulerian circuits.

Theorem 1.4. A graph G has an Euler circuit if and only if G is connected and every vertex of G has a positive even degree.

A related question:

- Given a graph G, is it possible to find a circuit for G in which all the vertices of G (except the first and the last) appear exactly once?

Hamiltonian Circuits (2/): history

- In 1859, the Irish mathematician Sir William Rowan Hamilton introduced a puzzle in the shape of a dodecahedron (DOH-dek-a-HEE-dron). (Figure 4 contains a drawing of a dodecahedron, which is a solid figure with 12 identical pentagonal faces.)

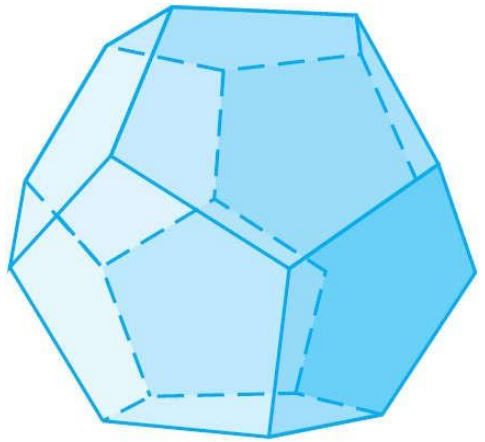


Figure 4 Dodecahedron

Hamiltonian Circuits (3/): history

- Each vertex was labeled with the name of a city — London, Paris, Hong Kong, New York, and so on.
- The problem Hamilton posed was to start at one city and tour the world by visiting each other city exactly once and returning to the starting city.

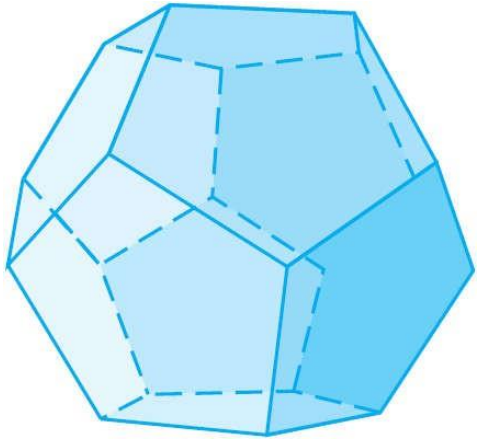
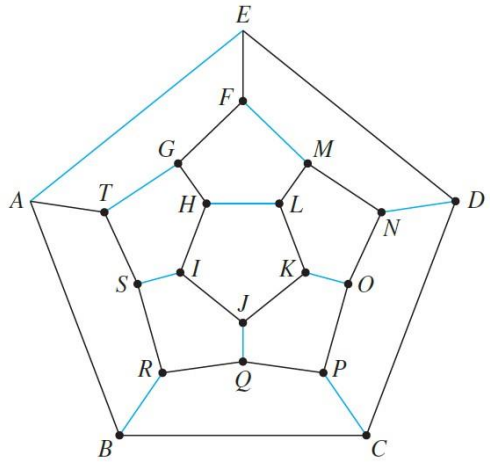


Figure 4 Dodecahedron

One way to solve the puzzle is to imagine the surface of the dodecahedron stretched out and laid flat in the plane



- One solution is the circuit

  A B C D E F G H I J K L M N O P Q R S T A,

  whose edges are indicated with black lines. Note that although every city is visited, many edges are omitted from the circuit.

Hamiltonian Circuits (4/): Definition

Definition 1. (Hamiltonian Circuit). Given a graph G, a Hamiltonian circuit for G is a simple circuit that includes every vertex of G.

That is, a Hamiltonian circuit for G is a sequence of adjacent vertices and distinct edges in which every vertex of G appears exactly once, except for the first and the last, which are the same.

- Note that although an Euler circuit for a graph G must include every edge of G, it may visit some vertices more than once and hence may not be a Hamiltonian circuit.
- On the other hand, a Hamiltonian circuit for G does not need to include all the edges of G and hence may not be an Euler circuit.
- Despite the analogous-sounding definitions of Euler and Hamiltonian circuits, the mathematics of the two are very different.

Hamiltonian Circuits (5/)

- Theorem 1. gives a simple criterion for determining whether a given graph has an Euler circuit.
- Unfortunately, there is no analogous criterion for determining whether a given graph has a Hamiltonian circuit, nor is there even an efficient algorithm for finding such a circuit.

Hamiltonian Circuits (6/): finding a Hamiltonian circuit

- However, a simple technique that can be used in many cases to show that a graph does *not* have a Hamiltonian circuit. This follows from the following considerations:

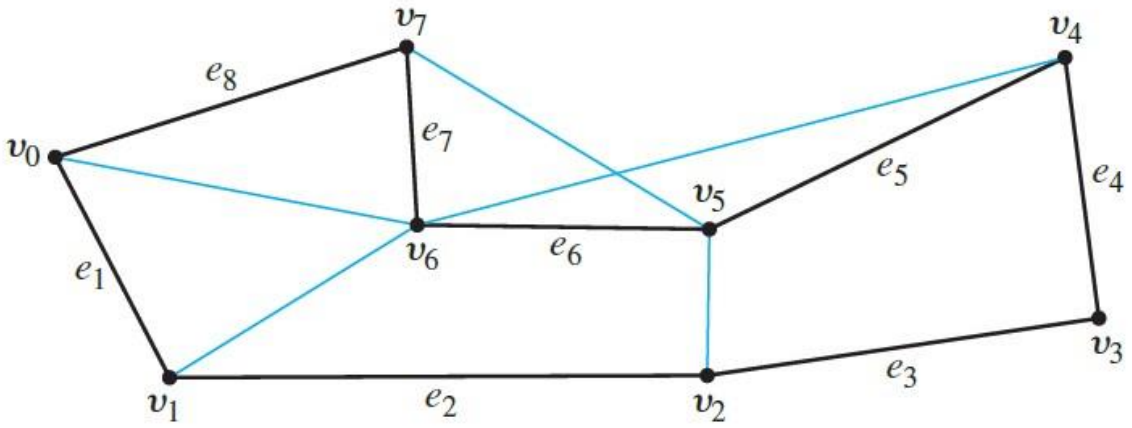
- Suppose a graph G with at least two vertices has a Hamiltonian circuit C, given concretely

$$C: v_0 e_1 v_1 e_2 \dots v_{n-1} e_n v_n (= v_0).$$

- Since C is a simple circuit, all the e_i are distinct and all the v_j are distinct except that $v_0 = v_n$.

Hamiltonian Circuits (7/): finding a Hamiltonian circuit

- Let H be the subgraph of G that is formed using the vertices and edges of C. An example of such an H is shown beside.



C: $v_0\, e_1 v_1 e_2\, v_2\, e_3 v_3\, e_4 v_4\, e_5 v_5\, e_6 v_6\, e_7 v_7 e_8\, v_0$

The edges of H are shown in black.

- Note that H has the same number of edges as it has vertices, since all its edges are distinct and so are its vertices.
- Also, by definition of Hamiltonian circuit, every vertex of G is a vertex of H, and H is connected since any two of its vertices lie on a circuit.
- Every vertex of H has degree 2. The reason for this is that there are exactly two edges incident on any vertex. These are e_i and e_{i+1} for any vertex v_j except $v_0 = v_n$, and they are e_1 and e_n for $v_0 (= v_n)$.

Hamiltonian Circuits (8/): finding a Hamiltonian circuit

- These observations have established the truth of the following proposition in all cases where G has at least two vertices.

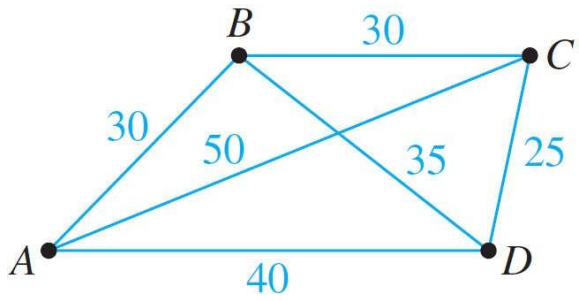
Proposition 1.. If a graph G has a Hamiltonian circuit, then G has a subgraph H with the following properties:

1. H contains every vertex of G.
2. H is connected.
3. H has the same number of edges as vertices.
4. Every vertex of H has degree 2.

The contrapositive of Proposition 1. says that if a graph G does not have a subgraph H with properties (1)–(4), then G does not have a Hamiltonian circuit.

Hamiltonian Circuits (9/): Travelling Salesman Problem

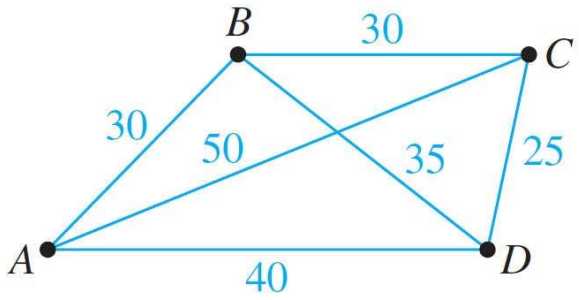
- Imagine that the drawing below is a map showing four cities and the distances in kilometers between them.



- Suppose that a salesman must travel to each city exactly once, starting and ending in city *A*. Which route from city to city will minimize the total distance that must be travelled?

Hamiltonian Circuits (10/): Travelling Salesman Problem: Solution

- This problem can be solved by writing all possible Hamiltonian circuits starting and ending at *A* and calculating the total distance travelled for each.



Route	Total Distance (In Kilometers)
$ABCDA$	$30 + 30 + 25 + 40 = 125$
$ABDCA$	$30 + 35 + 25 + 50 = 140$
$ACBDA$	$50 + 30 + 35 + 40 = 155$
$ACDBA$	140 [$ABDCA$ backwards]
$ADBCA$	155 [$ACBDA$ backwards]
$ADCBA$	125 [$ABCDA$ backwards]

- Thus either route *ABCDA* or *ADCBA* gives a minimum total distance of 125 km.

Hamiltonian Circuits (11/): Travelling Salesman Problem

- The general travelling salesman problem involves finding a Hamiltonian circuit to minimize the total distance travelled for an arbitrary graph with *n* vertices in which each edge is marked with a distance.

- One way to solve the general problem is to use the previous method: Write down all Hamiltonian circuits starting and ending at a particular vertex, compute the total distance for each, and pick one for which this total is minimal.

- However, this is impractical for even medium-sized values of *n*. For *n* = 30 vertices, there would be $(29!)/2 \approx 4.42 \times 10^{30}$ Hamiltonian circuits starting and ending at a particular vertex to check. If each circuit could be found and its total distance computed in just one nanosecond, it would take approximately 1.4×10^{14} years to compute!

Hamiltonian Circuits (12/): Travelling Salesman Problem

- At present, there is no known algorithm for solving the general travelling salesman problem that is more efficient.
  - This problem is an NP-complete problem.

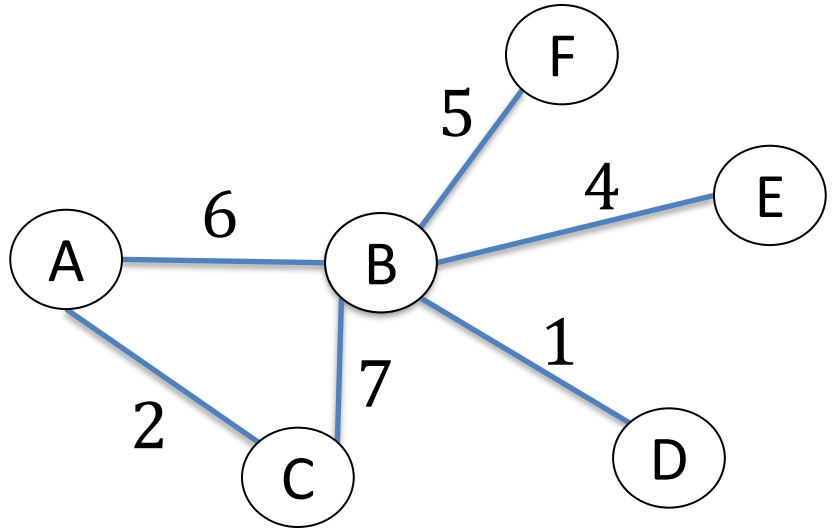
- However, there are efficient algorithms that find “pretty good” solutions—that is, circuits that, while not necessarily having the least possible total distances, have smaller total distances than most other Hamiltonian circuits.

Outline

1. Introduction
2. Definition and basic properties
3. Trails, paths, and circuits
4. Graph representations
5. Isomorphisms
6. Graph traversals
   - Breadth-First Search (BFS)
   - Depth-First Search (DFS)

Undirected Matrix representation: Adjacency matrix

Use a square matrix $A[n][n]$ with n is the number of vertices



$$A[i][j] = \begin{cases} w, \text{where } w \text{ is the weight of edge } (i,j) \\ 0, \text{if there is no edge } (i,j) \end{cases}$$

$$A[i][j] = \begin{cases} w, \text{where } w \text{ is the weight of edge } (i,j) \\ \infty, \text{if there is no edge } (i,j) \end{cases}$$

$$\begin{array}{c} \\ A \\ B \\ C \\ D \\ E \\ F \end{array} \begin{array}{c} \begin{array}{cccccc} A & B & C & D & E & F \end{array} \\ \begin{bmatrix} 0 & 6 & 2 & 0 & 0 & 0 \\ 6 & 0 & 7 & 1 & 4 & 5 \\ 2 & 7 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 4 & 0 & 0 & 0 & 0 \\ 0 & 5 & 0 & 0 & 0 & 0 \end{bmatrix} \end{array}$$

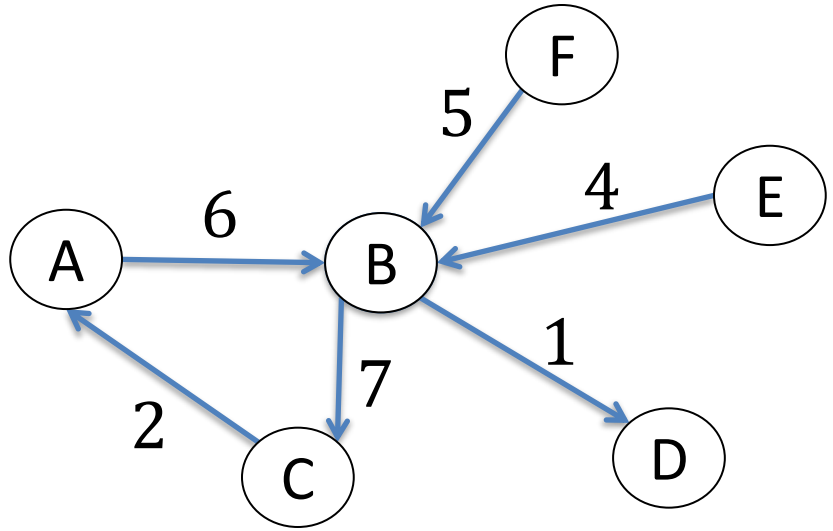
Space costs $|V|^2$

Matrix is SYMMETRIC

$$\begin{array}{c} \\ A \\ B \\ C \\ D \\ E \\ F \end{array} \begin{array}{c} \begin{array}{cccccc} A & B & C & D & E & F \end{array} \\ \begin{bmatrix} \infty & 6 & 2 & \infty & \infty & \infty \\ 6 & \infty & 7 & 1 & 4 & 5 \\ 2 & 7 & \infty & \infty & \infty & \infty \\ \infty & 1 & \infty & \infty & \infty & \infty \\ \infty & 4 & \infty & \infty & \infty & \infty \\ \infty & 5 & \infty & \infty & \infty & \infty \end{bmatrix} \end{array}$$

Directed Matrix representation: Adjacency matrix

Use a square matrix $A[n][n]$ with n is the number of vertices



$$A[i][j] = \begin{cases} w, \text{where } w \text{ is the weight of edge } (i,j) \\ 0, \text{if there is no edge } (i,j) \end{cases}$$

$$A[i][j] = \begin{cases} w, \text{where } w \text{ is the weight of edge } (i,j) \\ \infty, \text{if there is no edge } (i,j) \end{cases}$$

$$\begin{array}{c} \\ A \\ B \\ C \\ D \\ E \\ F \end{array} \begin{array}{c} \begin{array}{cccccc} A & B & C & D & E & F \end{array} \\ \begin{bmatrix} 0 & 6 & 0 & 0 & 0 & 0 \\ 0 & 0 & 7 & 1 & 0 & 0 \\ 2 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 4 & 0 & 0 & 0 & 0 \\ 0 & 5 & 0 & 0 & 0 & 0 \end{bmatrix} \end{array}$$

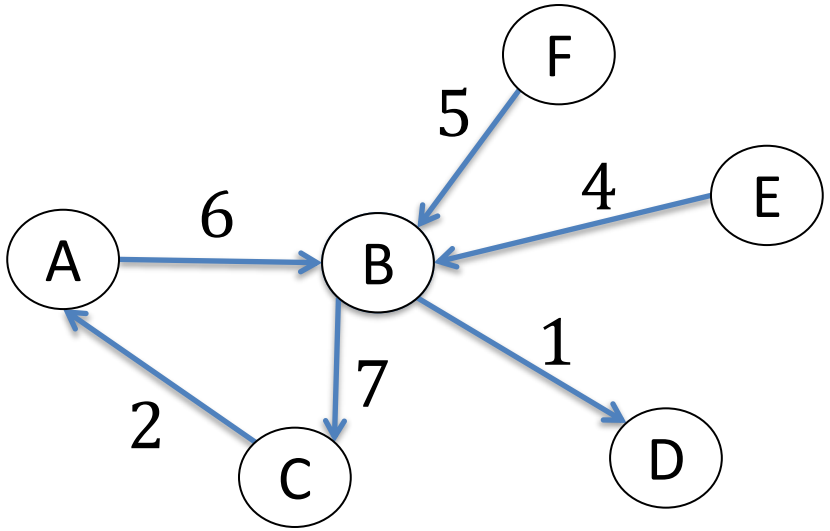
Space costs $|V|^2$

Matrix might be ASYMMETRIC

$$\begin{array}{c} \\ A \\ B \\ C \\ D \\ E \\ F \end{array} \begin{array}{c} \begin{array}{cccccc} A & B & C & D & E & F \end{array} \\ \begin{bmatrix} \infty & 6 & \infty & \infty & \infty & \infty \\ \infty & \infty & 7 & 1 & \infty & \infty \\ 2 & \infty & \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty & \infty & \infty \\ \infty & 4 & \infty & \infty & \infty & \infty \\ \infty & 5 & \infty & \infty & \infty & \infty \end{bmatrix} \end{array}$$

Directed Matrix representation: Adjacency matrix

Use a square matrix $A[n][n]$ with n is the number of vertices



$$A[i][j] = \begin{cases} w, \text{where } w \text{ is the number of arrows from i to j} \\ 0, \text{if there is no edge } (i,j) \end{cases}$$

$$\begin{array}{c} \\ A \\ B \\ C \\ D \\ E \\ F \end{array} \begin{array}{c} \begin{array}{cccccc} A & B & C & D & E & F \end{array} \\ \begin{bmatrix} 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \end{bmatrix} \end{array}$$

Space costs $|V|^2$

Matrix might be ASYMMETRIC

Undirected Matrix representation: Adjacency list

Use $n = 6$ lists representing for connected vertices to a vertex

A→C,2→B,6

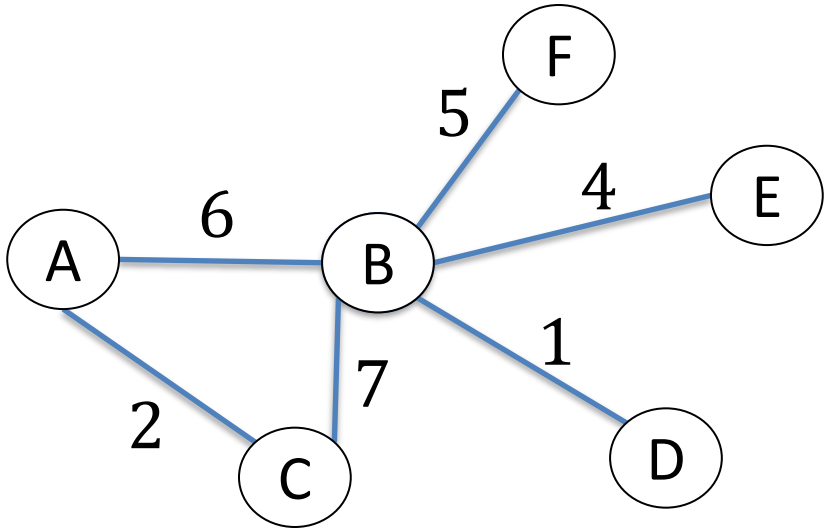
B→A,6→C,7→D,1→E,4→F,5

C→A,2→B,7

D→B,1

E→B,4

F→B,5



Space costs $|V| + |E|$

Each row contains an edge and its weight

A C 2

A B 6

B C 7

B D 1

B E 4

B F 5

Directed Matrix representation: Adjacency list

Use $n = 6$ lists representing for connected vertices to a vertex

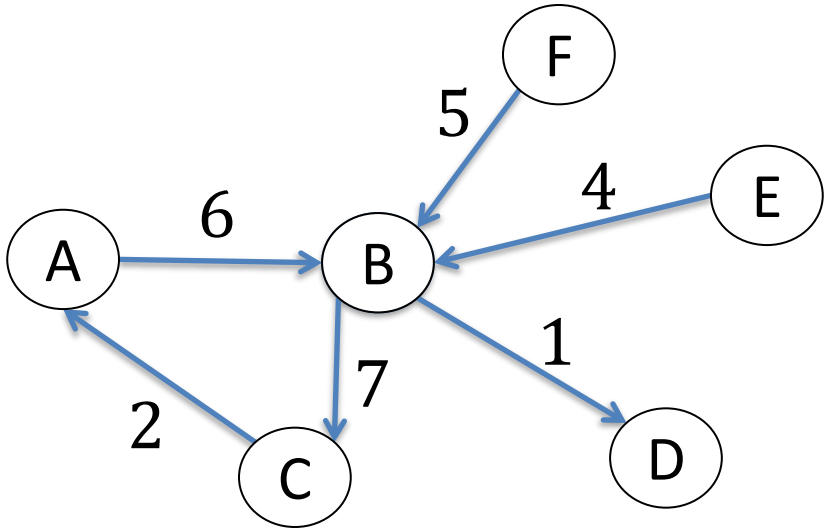
A→B,6

B→C,7→D,1

C→A,2

E→B,4

F→B,5



Space costs $|V| + |E|$

Each row contains an edge and its weight

A B 6

B C 7

C A 2

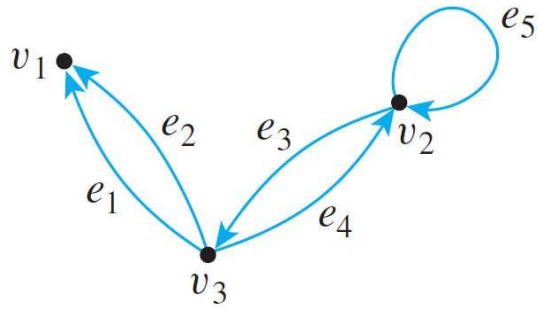
B D 1

E B 4

F B 5

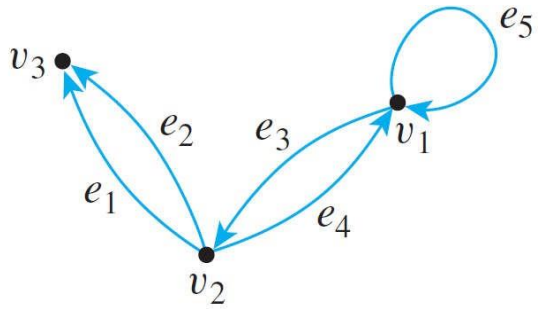
Directed Matrix representation: Adjacency list

Find the adjacency matrices of the two directed graphs below.



$$\mathbf{A} = \begin{array}{c} \\ v_1 \\ v_2 \\ v_3 \end{array} \begin{array}{c} \begin{array}{ccc} v_1 & v_2 & v_3 \end{array} \\ \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 1 \\ 2 & 1 & 0 \end{bmatrix} \end{array}$$

(a)

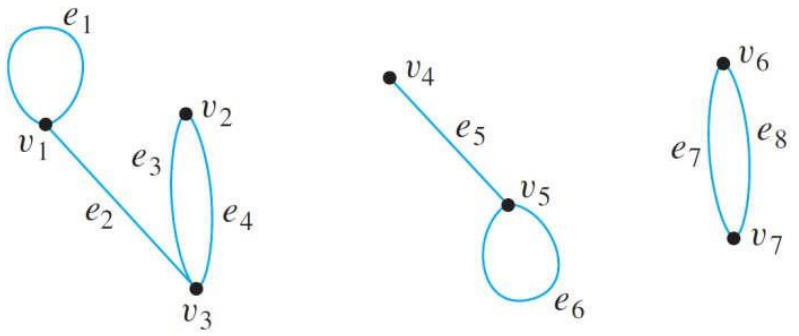


$$\mathbf{A} = \begin{array}{c} \\ v_1 \\ v_2 \\ v_3 \end{array} \begin{array}{c} \begin{array}{ccc} v_1 & v_2 & v_3 \end{array} \\ \begin{bmatrix} 1 & 1 & 0 \\ 1 & 0 & 2 \\ 0 & 0 & 0 \end{bmatrix} \end{array}$$

(b)

Matrices and Connected Components

- Consider a graph *G*, as shown below, that consists of several connected components.



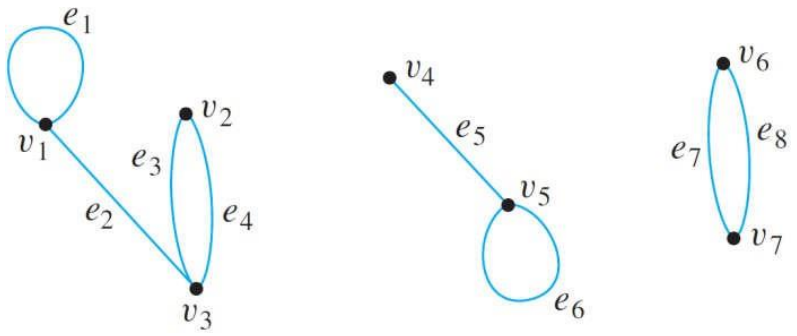
Adjacency matrix of G

$$\mathbf{A} = \left[\begin{array}{ccc:ccc:cc} 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 2 & 0 & 0 & 0 & 0 \\ 1 & 2 & 0 & 0 & 0 & 0 & 0 \\ \hdashline 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 & 0 \\ \hdashline 0 & 0 & 0 & 0 & 0 & 0 & 2 \\ 0 & 0 & 0 & 0 & 0 & 2 & 0 \end{array}\right]$$

- As you can see, **A** consists of square matrix blocks (of different sizes) down its diagonal and blocks of 0's everywhere else.

Matrices and Connected Components

- The reason is that vertices in each connected component share no edges with vertices in other connected components.



Adjacency matrix of G

$$\mathbf{A} = \left[\begin{array}{ccc:ccc:cc} 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 2 & 0 & 0 & 0 & 0 \\ 1 & 2 & 0 & 0 & 0 & 0 & 0 \\ \hdashline 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 & 0 \\ \hdashline 0 & 0 & 0 & 0 & 0 & 0 & 2 \\ 0 & 0 & 0 & 0 & 0 & 2 & 0 \end{array}\right]$$

- For instance, since v_1, v_2, and v_3 share no edges with v_4, v_5, v_6, or v_7, all entries in the top three rows to the right of the third column are 0 and all entries in the left three columns below the third row are also 0.

Matrices and Connected Components

- Sometimes matrices whose entries are all 0's are themselves denoted 0. If this convention is followed here, A is written as:

$$\mathbf{A} = \left[\begin{array}{ccc|c|c} 1 & 0 & 1 & & \\ 0 & 0 & 2 & & \\ 1 & 2 & 0 & & \\ \hline & & & \begin{matrix} 0 & 1 \\ 1 & 1 \end{matrix} & \\ \hline & & & & \begin{matrix} 0 & 2 \\ 2 & 0 \end{matrix} \end{array}\right]$$

Adjacency matrix of G in the form of block diagonal matrices

Matrices and Connected Components

- The previous reasoning can be generalized to prove the following theorem:

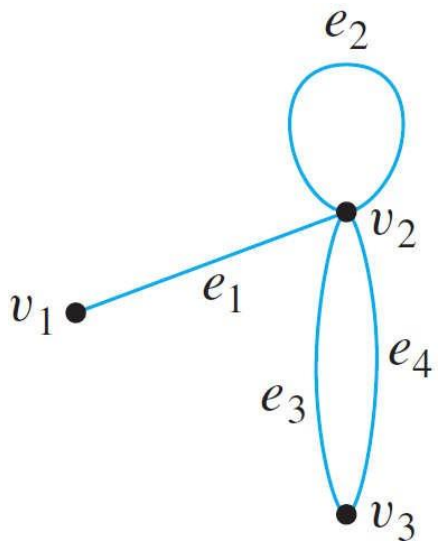
Theorem 1.. Let G be a graph with connected components $G_1, G_2, \ldots, G_k$. If there are n_i vertices in each connected component G_i and these vertices are numbered consecutively, then the adjacency matrix of G has the form:

$$\begin{bmatrix} A_1 & 0 & 0 & \ldots & 0 & 0 \\ 0 & A_2 & 0 & \ldots & 0 & 0 \\ 0 & 0 & A_3 & \ldots & 0 & 0 \\ \vdots & \vdots & \vdots & & \vdots & \vdots \\ 0 & 0 & 0 & \ldots & 0 & A_k \end{bmatrix}$$

where each A_i is $n_i \times n_i$ adjacency matrix of G_i, for all $i = 1, 2, \ldots, k$, and the O's represent matrices whose entries are all 0s.

Counting Walks of Length N

- Example: Consider the following graph G.
- How many distinct walks of length 2 connect v_2 and v_2?



- One walk of length 2 from v_2 to v_2 that starts by going from v_2 to v_1:

$$v_2 e_1 v_1 e_1 v_2.$$

- One walk of length 2 from v_2 to v_2 that starts by going from v_2 to v_2:

$$v_2 e_2 v_2 e_2 v_2.$$

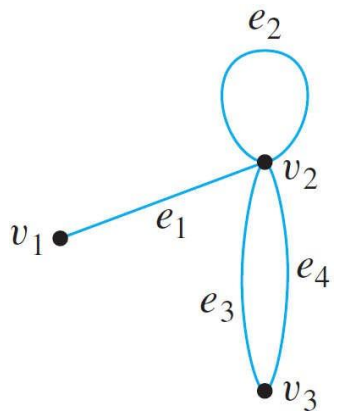
- Four walks of length 2 from v_2 to v_2 that start by going from v_2 to v_3:

$$v_2 e_3 v_3 e_4 v_2, v_2 e_4 v_3 e_3 v_2, v_2 e_3 v_3 e_3 v_2, v_2 e_4 v_3 e_4 v_2.$$

- Thus the answer is six.

Counting Walks of Length N

- The general question of finding the number of walks that have a given length and connect two particular vertices of a graph can easily be answered using matrix multiplication.
- Consider the adjacency matrix A of the graph G.



$$\mathbf{A} = \begin{array}{c} \\ v_1 \\ v_2 \\ v_3 \end{array} \begin{array}{c} \begin{array}{ccc} v_1 & v_2 & v_3 \end{array} \\ \begin{bmatrix} 0 & 1 & 0 \\ 1 & 1 & 2 \\ 0 & 2 & 0 \end{bmatrix} \end{array}.$$

Counting Walks of Length N

- Compute A^2

$$A^2 = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 1 & 2 \\ 0 & 2 & 0 \end{bmatrix} \begin{bmatrix} 0 & 1 & 0 \\ 1 & 1 & 2 \\ 0 & 2 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 2 \\ 1 & 6 & 2 \\ 2 & 2 & 4 \end{bmatrix}$$

- Note that the entry in the second row and the second column is 6, which equals the number of walks of length 2 from v_2 to v_2. This is no accident!
- To compute a_{22}, you multiply the second row of A times the second column of A to obtain a sum of three terms:

$$\begin{bmatrix} 1 & 1 & 2 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \\ 2 \end{bmatrix} = 1 \cdot 1 + 1 \cdot 1 + 2 \cdot 2 = 6.$$

Counting Walks of Length N

- Observe that

$$\left[\begin{matrix}\text{the first term}\\ \text{of this sum}\end{matrix}\right] = \left[\begin{matrix}\text{number of edges}\\ \text{from } v_2 \text{ to } v_1\end{matrix}\right] \cdot \left[\begin{matrix}\text{number of edges}\\ \text{from } v_1 \text{ to } v_2\end{matrix}\right]$$

$$= \text{number of pairs of edges from } \mathrm{v}_2 \text{ to } \mathrm{v}_1 \text{ and } \mathrm{v}_1 \text{to } \mathrm{v}_2$$

Matrices and Connected Components

- More generally, if A is the adjacency matrix of a graph *G*, the *i j*-th entry of A^2 equals the number of walks of length 2 connecting the *i*-th vertex to the *j*-th vertex of *G*.
- Even more generally, if *n* is any positive integer, the *i j*-th entry of A^n equals the number of walks of length *n* connecting the *i*-th and the *j*-th vertices of *G*.

Theorem 1.. If G is a graph with vertices $v_1, v_2, \dots, v_m$ and A is the adjacency matrix of G, then for each positive integer n and for all integers $i, j = 1, 2, \dots, m$,

the ij-th entry of A^n = the number of walks of length n from v_i to v_j.

Outline

1. Introduction
2. Definition and basic properties
3. Trails, paths, and circuits
4. Graph representations
5. Isomorphisms
6. Graph traversals
   - Breadth-First Search (BFS)
   - Depth-First Search (DFS)

Introduction

- The two following drawings both represent the same graph: Their vertex and edge sets are identical, and their edge-endpoint functions are the same. Call this graph G.

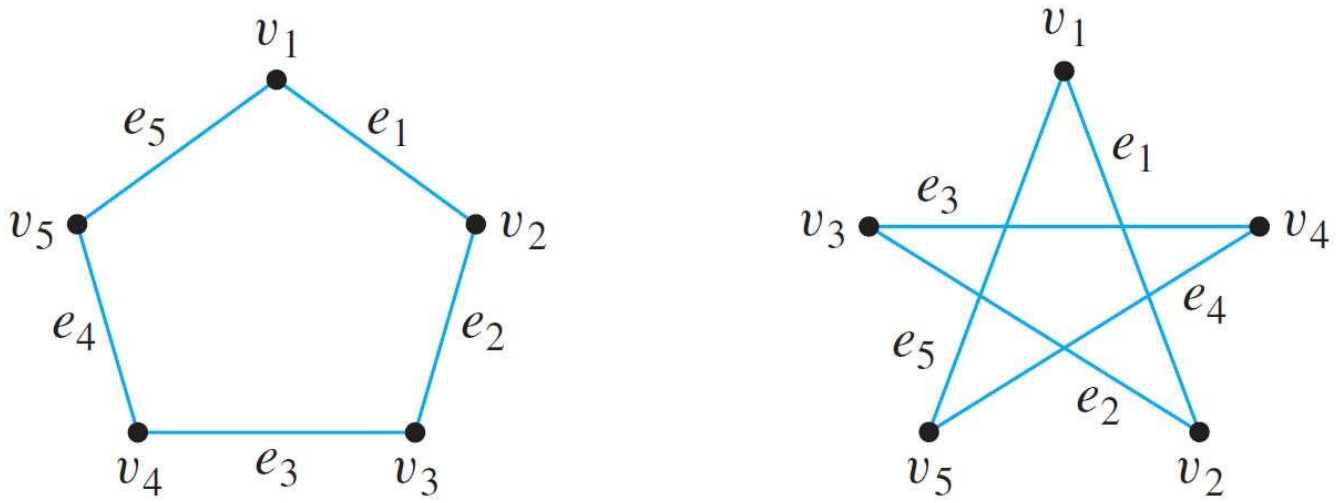


Figure 1.

Introduction

- Now consider the graph G' represented in Figure 2.

- Observe that G' is a different graph from G (for instance, in G the endpoints of e_1 are v_1 and v_2, whereas in G' the endpoints of e_1 are v_1 and v_3).

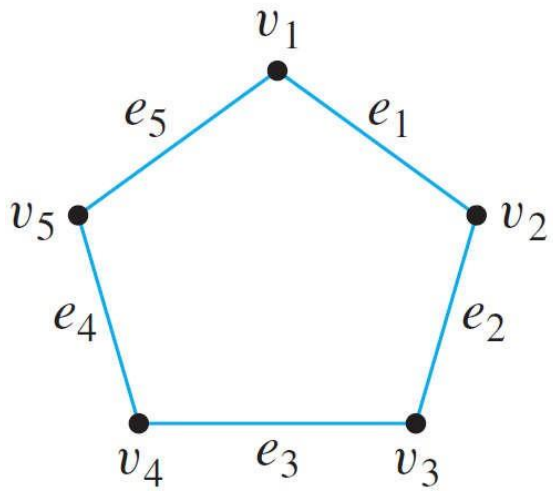
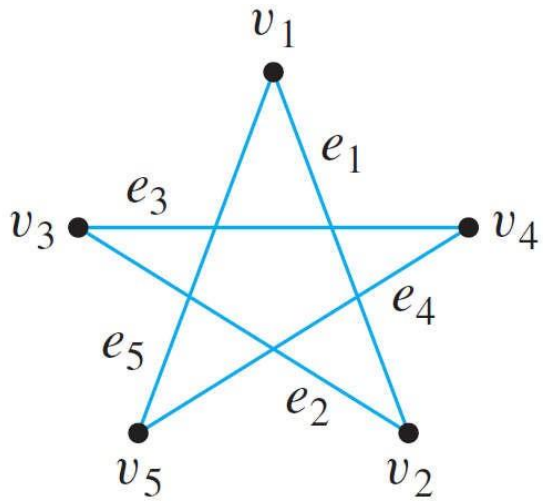


Figure 1.



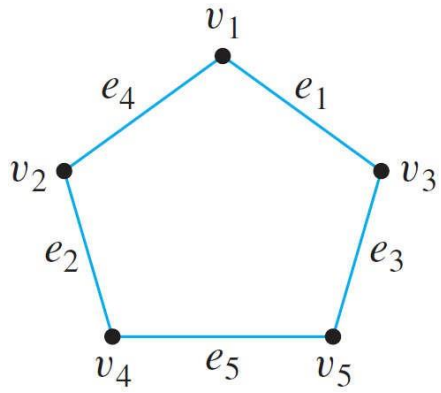


Figure 2.

Introduction

- Yet G' is certainly very similar to G. In fact, if the vertices and edges of G' are relabeled by the functions shown in Figure 3, then G' becomes the same as G.

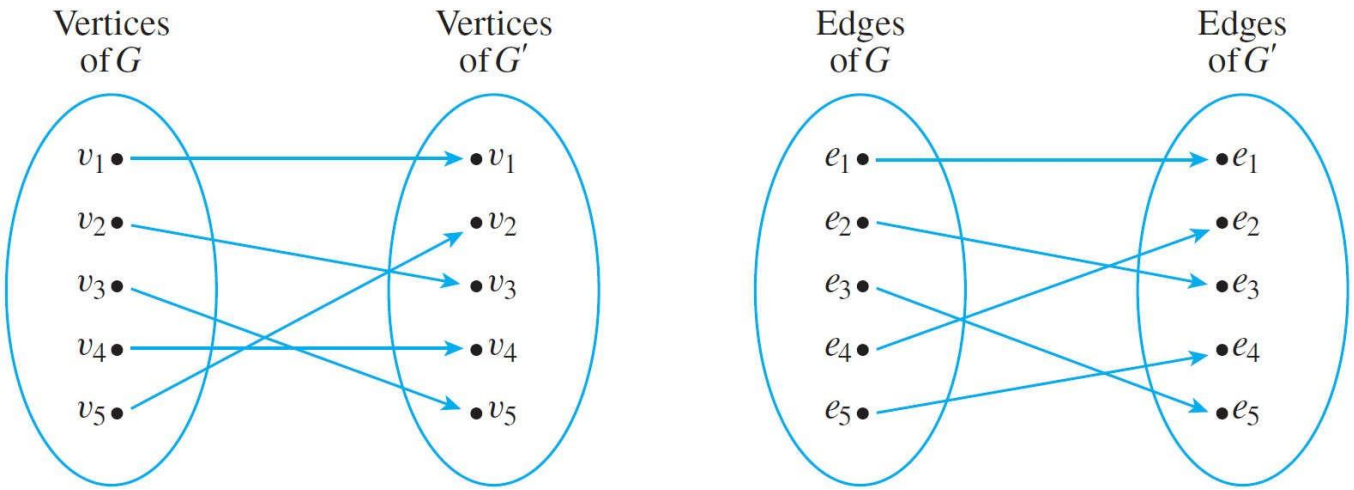


Figure 3.

- Note that these relabeling functions are **one-to-one** and **onto**.

Definition

Definition 1. Let G and G' be graphs with vertex sets $V(G)$ and $V(G')$ and edge sets $E(G)$ and $E(G')$ respectively.

G is isomorphic to G‘ if and only if there exist one-to-one correspondences $g: V(G) \rightarrow V(G')$ and $h: E(G) \rightarrow E(G')$ that preserve the edge-endpoint functions of G and G‘ in the sense that for all $v \in V(G)$ and $e \in E(G)$, v is an endpoint of $e \Leftrightarrow g(v)$ is an endpoint of $h(e)$.

Graph Isomorphism for Simple Graphs

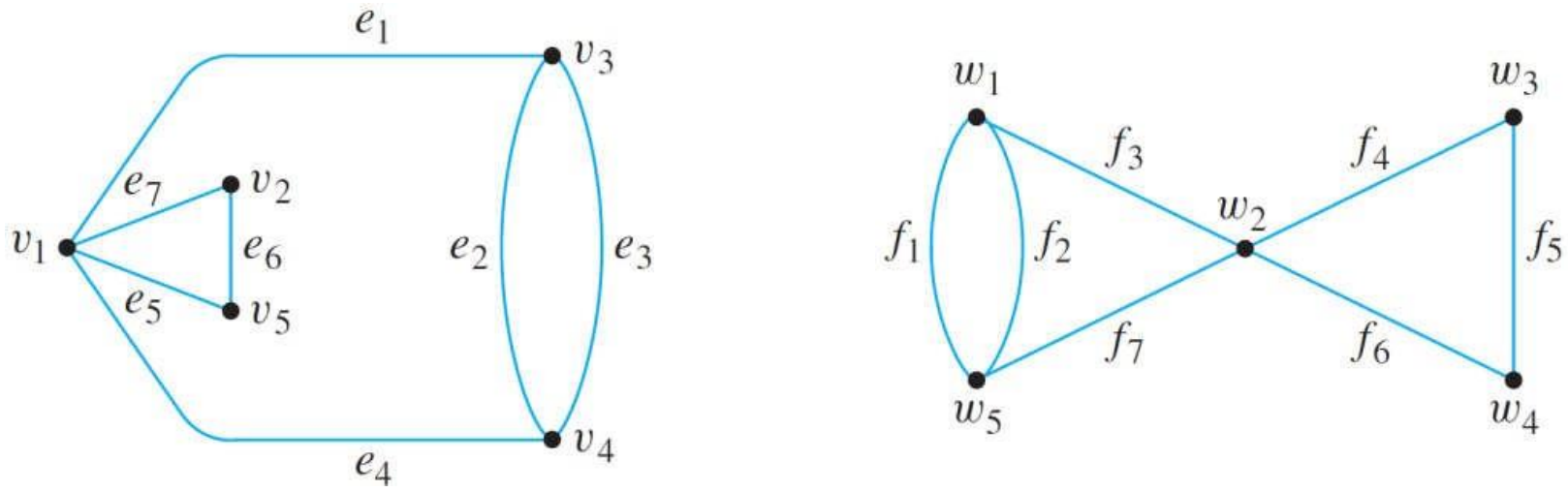
- When graphs G and G' are both simple, the definition of G being isomorphic to G' can be written without referring to the correspondence between the edges of G and the edges of G'.

Definition 1. If G and G' are simple graphs, then G is isomorphic to G' if and only if there exists a one-to-one correspondence g from the vertex set $V(G)$ of G to the vertex set $V(G')$ of G' that preserves the edge-endpoint functions of G and G' in the sense that for all vertices u and v of G,

$\{u, v\}$ is an edge in $G \iff \{g(u), g(v)\}$ is an edge in G'.

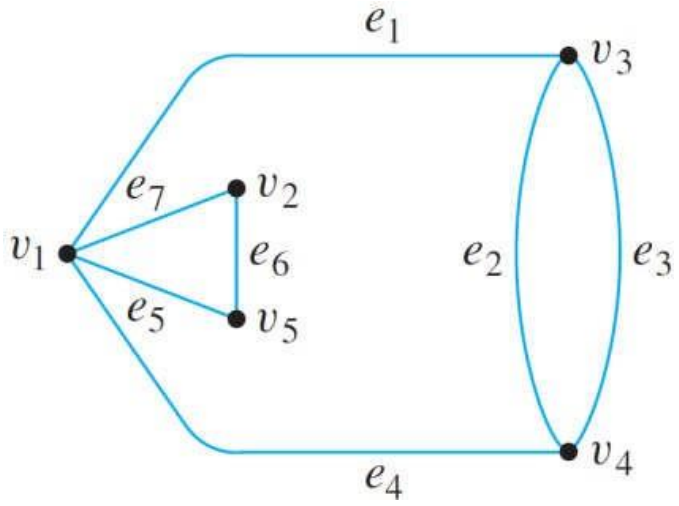
Example

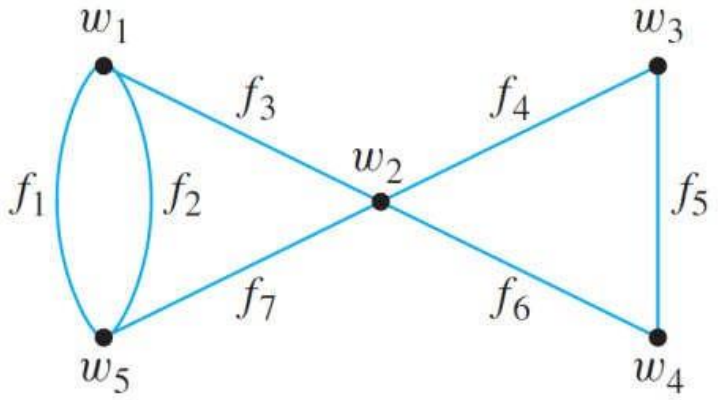
- Show that the following two graphs are isomorphic.

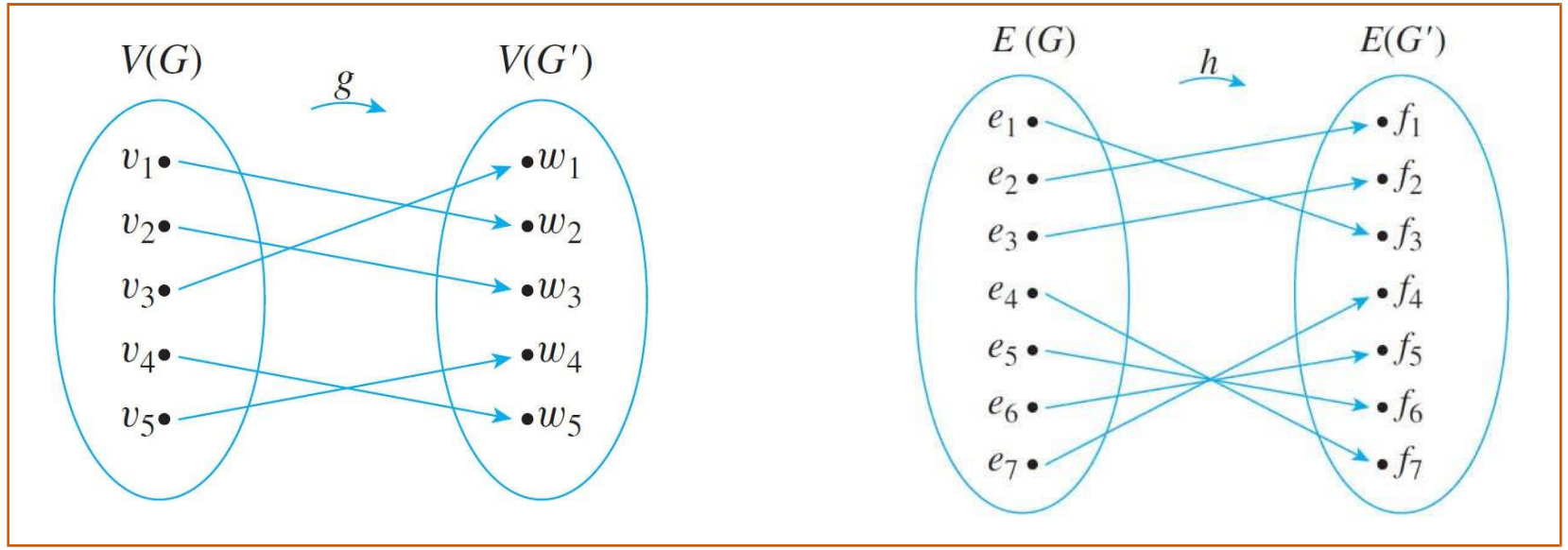


To solve this, you find functions $g: V(G) \to V(G')$ and $h: E(G) \to E(G')$ such that for all for all $v \in V(G)$ and $e \in E(G)$, v is an endpoint of e if, and only if, $g(v)$ is an endpoint of $h(e)$.

Solution


e_1
v_3
e_7
v_2
v_1
e_6
e_2
e_3
e_5
v_5
e_4
v_4


w_1
w_3
f_3
f_4
w_2
f_1
f_2
f_5
f_7
f_6
w_5
w_4


$V(G)$
g
$V(G')$
v_1
v_2
v_3
v_4
v_5
w_1
w_2
w_3
w_4
w_5
$E(G)$
h
$E(G')$
e_1
e_2
e_3
e_4
e_5
e_6
e_7
f_1
f_2
f_3
f_4
f_5
f_6
f_7

Graph Isomorphism is an Equivalence Relation

Theorem 1.. Let S be a set of graphs and let R be the relation of graph isomorphism on S.

Then R is an equivalence relation on S.

- In other words, the graph isomorphism is reflexive, symmetric, and transitive.

Checking whether two graphs are isomorphic (1/)

- Is there a general method (algorithm) to determine if two graphs *G* and *G'* are isomorphic?

There is such an algorithm, but it takes an unreasonably long time to perform.

- If *G* and *G'* each has *n* vertices and *m* edges, the number of one-to-one correspondences from vertices to vertices is *n*! and the number of one-to-one correspondences from edges to edges is *m*!, so the total number of pairs of functions to check is *n*!*m*!.

For example, if $m = n = 25$, there would be $25!\,25! \approx 2.4 \times 10^{50}$ pairs to check. If each check takes 1 ns, the total time would be approximately 7.6×10^{33} years (7.6 million billion billion billion years)!

Checking whether two graphs are isomorphic (2/)

- Unfortunately, there is no more efficient general method known for checking whether two graphs are isomorphic.
- However, there are some simple tests that can be used to show that certain pairs of graphs are not isomorphic.
- For instance, if two graphs are isomorphic, then they have the same number of vertices (because there is a one-to-one correspondence from the vertex set of one graph to the vertex set of the other).
  - It follows that if you are given two graphs, one with 16 vertices and the other with 17, you can immediately conclude that the two are not isomorphic.
- More generally, a property that is preserved by graph isomorphism is called an isomorphic invariant.

Definition

Definition 1. A property P is called an invariant for graph isomorphism if and only if given any graphs G and G', if G has property P and G' is isomorphic to G, then G' has property P.

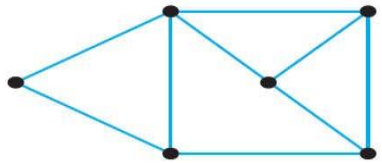
Theorem 1.. Each of the following properties is an invariant for graph isomorphism, where n, m, and k are all non-negative integers.

1. has n vertices
2. has a vertex of degree k
3. has a circuit of length k
4. has m simple circuits of length k
5. has an Euler circuit
6. has m edges
7. has m vertices of degree k
8. has a simple circuit of length k
9. is connected
10. has a Hamiltonian circuit.

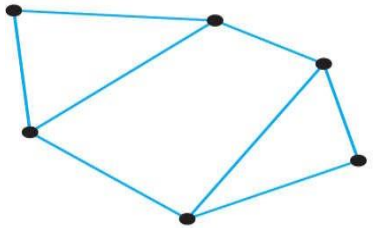
Quiz

- Show that the following pairs of graphs are not isomorphic by finding an isomorphic invariant that they do not share.

a.

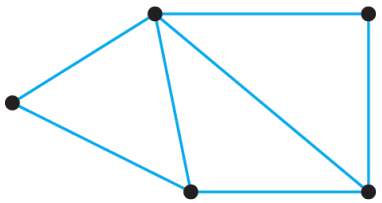


G

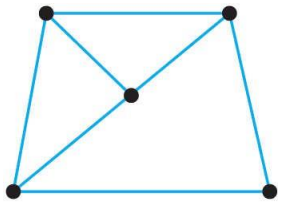


G'

b.



H



H'

Graph Isomorphism for Simple Graphs

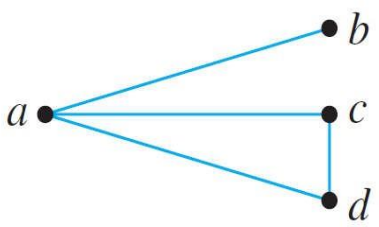
- When graphs G and G' are both simple, the definition of G being isomorphic to G' can be written without referring to the correspondence between the edges of G and the edges of G'.

Definition 1. If G and G' are simple graphs, then G is isomorphic to G' if and only if there exists a one-to-one correspondence g from the vertex set $V(G)$ of G to the vertex set $V(G')$ of G' that preserves the edge-endpoint functions of G and G' in the sense that for all vertices u and v of G,

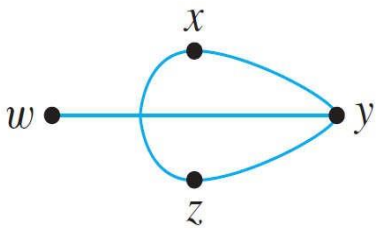
$\{u, v\}$ is an edge in $G \iff \{g(u), g(v)\}$ is an edge in G'.

Example

- Are the two graphs shown below isomorphic? If so, define an isomorphism.

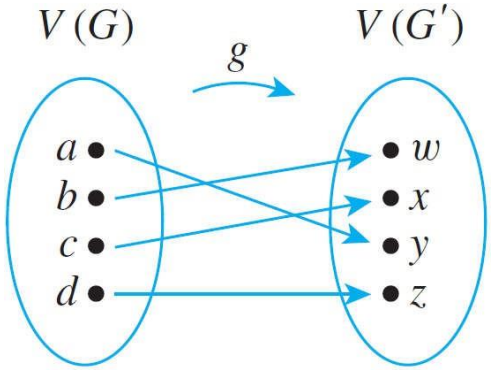


G



G'

Yes. Define $g: V(G) \to V(G')$ by the arrow diagram shown below. Then g is one-to-one and onto by inspection.

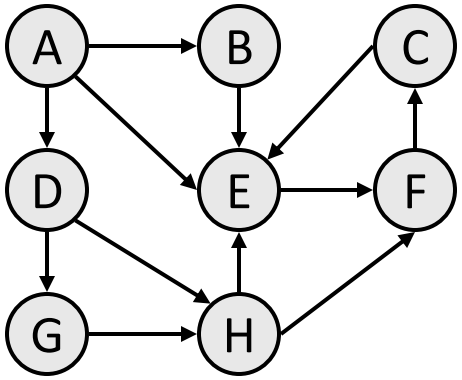


Edges of G	Edges of G'
$\{a, b\}$	$\{y, w\} = \{g(a), g(b)\}$
$\{a, c\}$	$\{y, x\} = \{g(a), g(c)\}$
$\{a, d\}$	$\{y, z\} = \{g(a), g(d)\}$
$\{c, d\}$	$\{x, z\} = \{g(c), g(d)\}$

Outline

1. Introduction
2. Definition and basic properties
3. Trails, paths, and circuits
4. Graph representations
5. Isomorphisms
6. Graph traversals
   - Breadth-First Search (BFS)
   - Depth-First Search (DFS)

- **Breadth-First Search** (BFS): Take one step down all paths and immediately backtrack.
  - A queue of vertices to visit.
  - Always returns the shortest path (one with the fewest edges):
    - to B: {A, B}
    - to C: **{A, E, F, C}**
    - to D: {A, D}
    - to E: **{A, E}**
    - to F: **{A, E, F}**
    - to G: {A, D, G}
    - to H: **{A, D, H}**



- **Depth-First Search** (DFS): Explore each possible path as far as possible before backtracking.
  - Often implemented recursively.
  - DFS paths from A to all vertices (assuming ABC edge order):
    - to B: {A, B}
    - to C: {A, B, E, F, C}
    - to D: {A, D}
    - to E: {A, B, E}
    - to F: {A, B, E, F}
    - to G: {A, D, G}
    - to H: {A, D, G, H}

Outline

1. Introduction
2. Definition and basic properties
3. Trails, paths, and circuits
4. Graph representations
5. Isomorphisms
6. Graph traversals
   - Breadth-First Search (BFS)
   - Depth-First Search (DFS)

BFS algorithm

BFS(G, start)

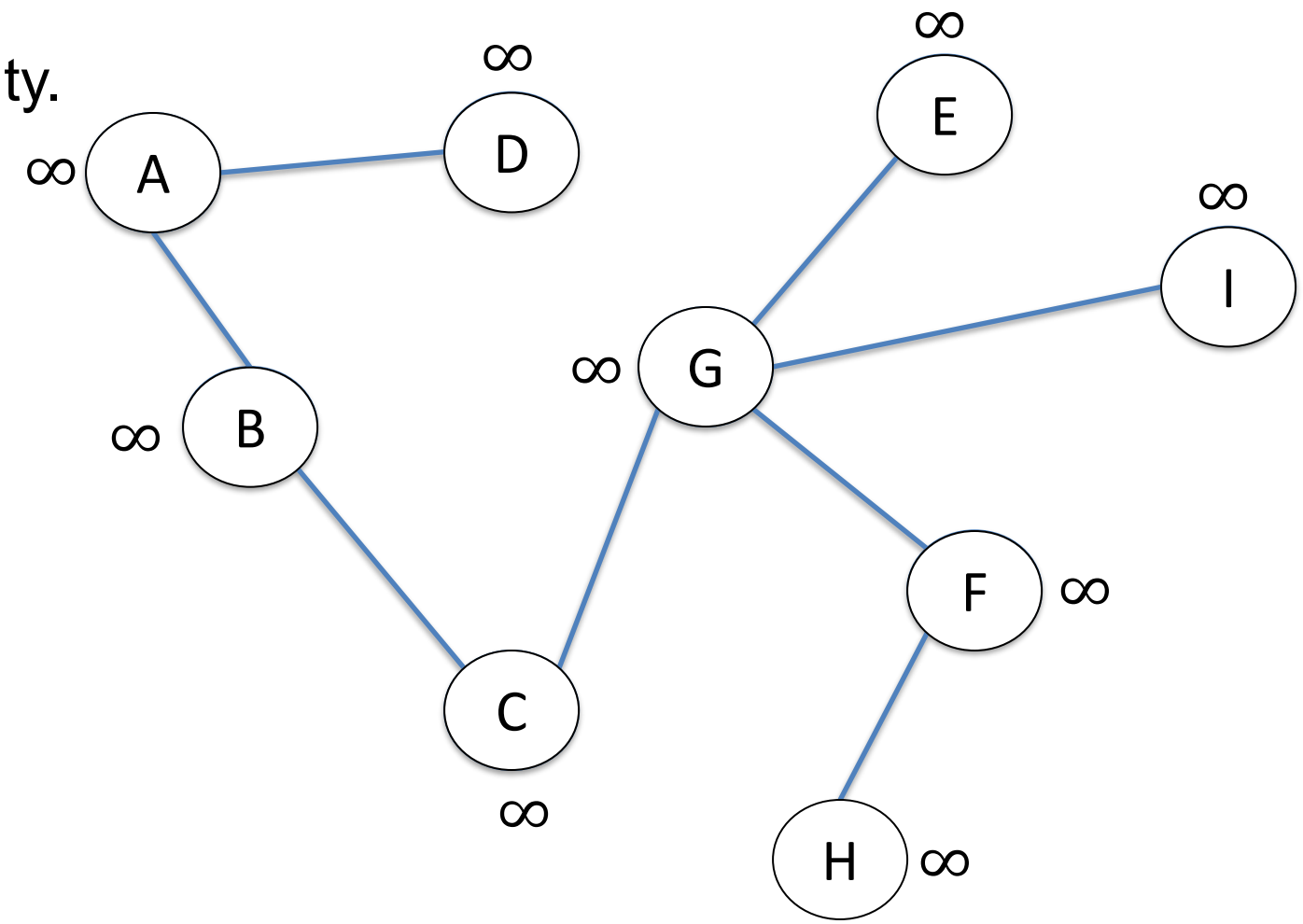
```
Create a new queue Q
Q.push(start)
dist[1..n] = {∞, …, ∞}
dist[start] = 0

while Q is not empty
    u = Q.pop()
    for each node v adjacent to u
        if dist[v] = ∞ then
            dist[v] = dist[u] + 1
            Q.push(v)
```

Suppose we have a graph $G = (V, E)$ containing $|V| = n$ nodes and $|E| = m$ edges.

BFS takes $O(n + m)$ time and space.

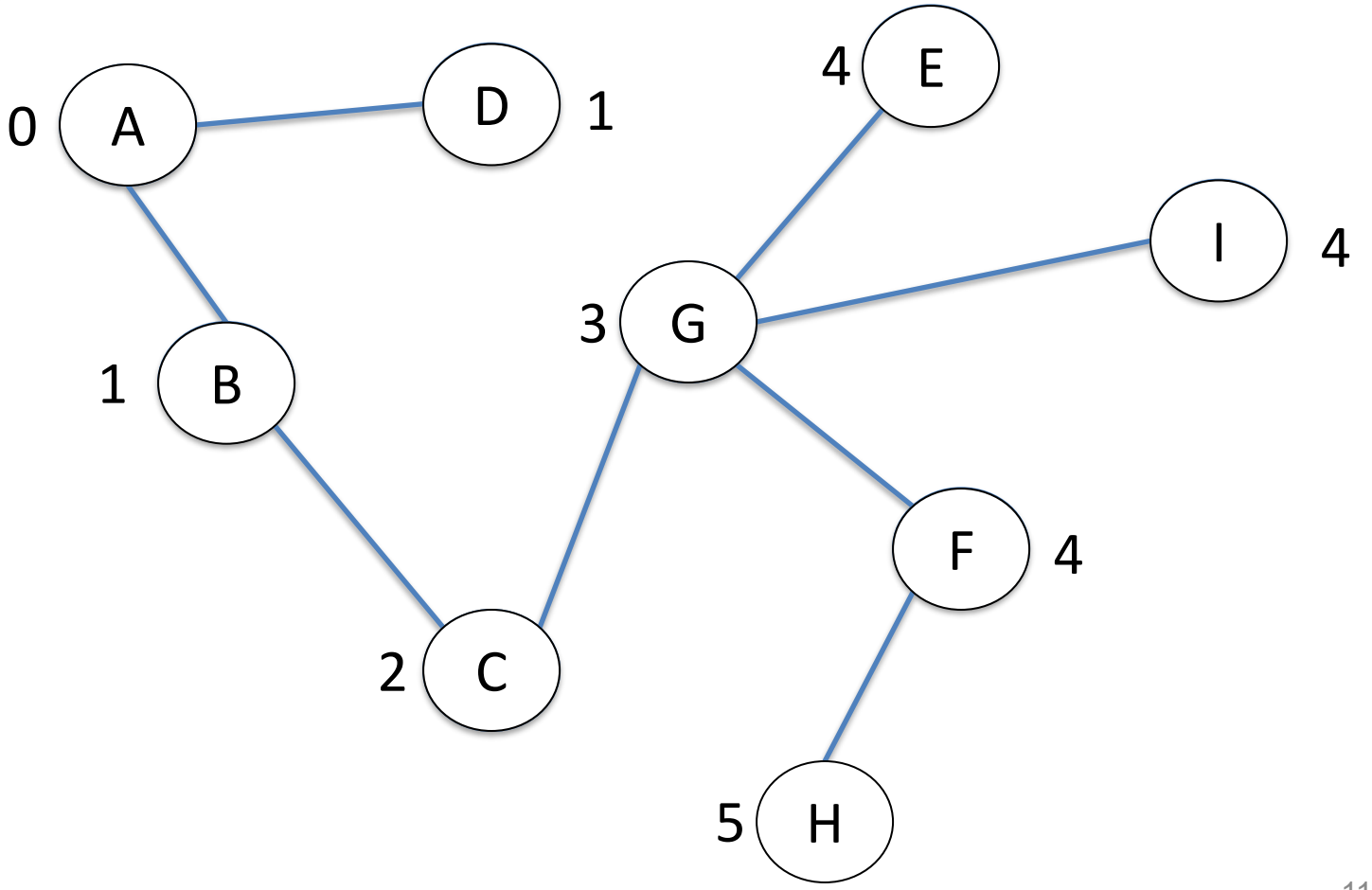
Example

- BFS starting at node A.
- All weights initially set to infinity.



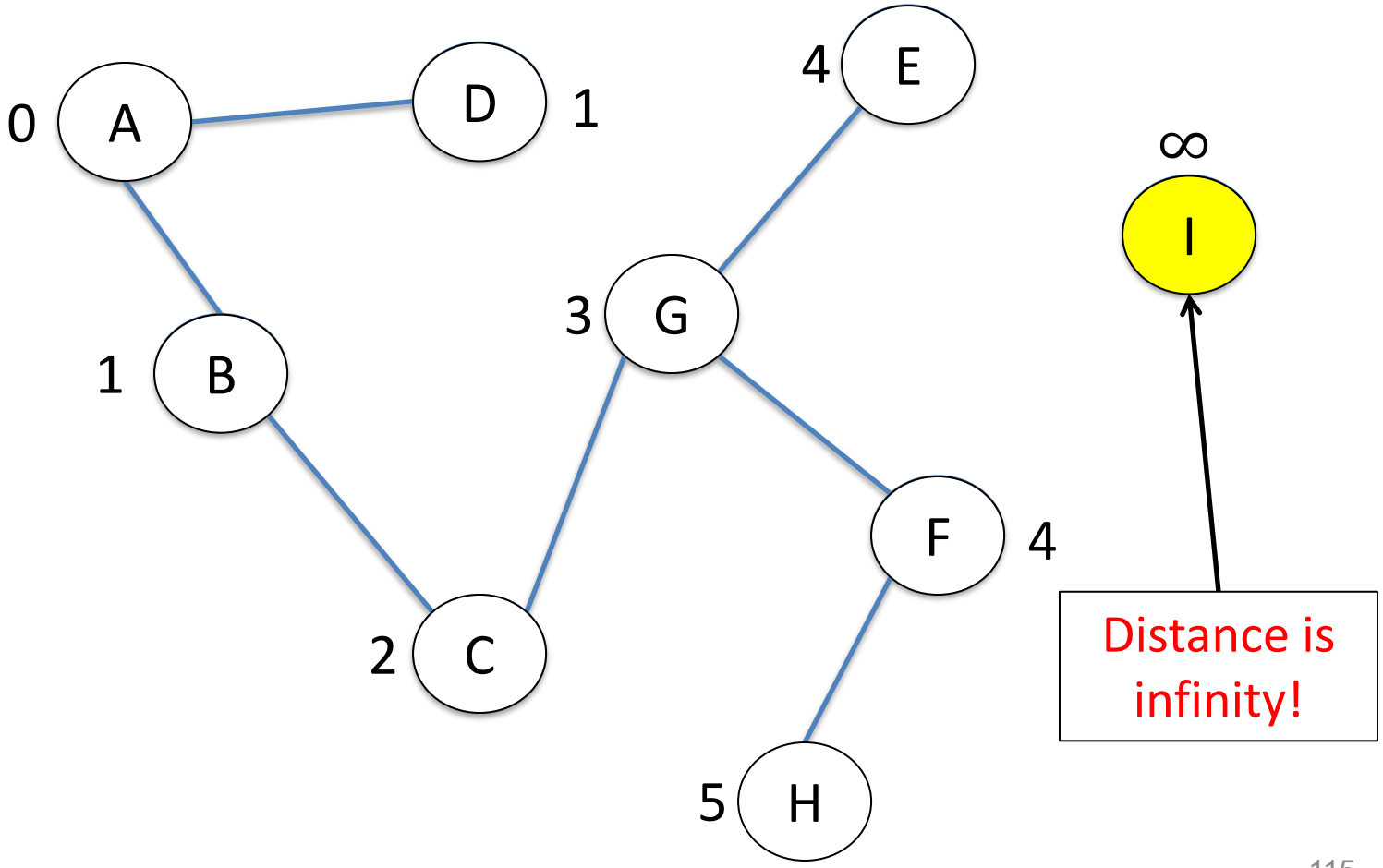
Application 1: checking connectedness (1/2)

- Run BFS on any node
- If no node has distance infinity, it's **connected**!



Application 1: checking connectedness (2/2)

- Run BFS on any node
- If no node has distance infinity, it's **connected**!



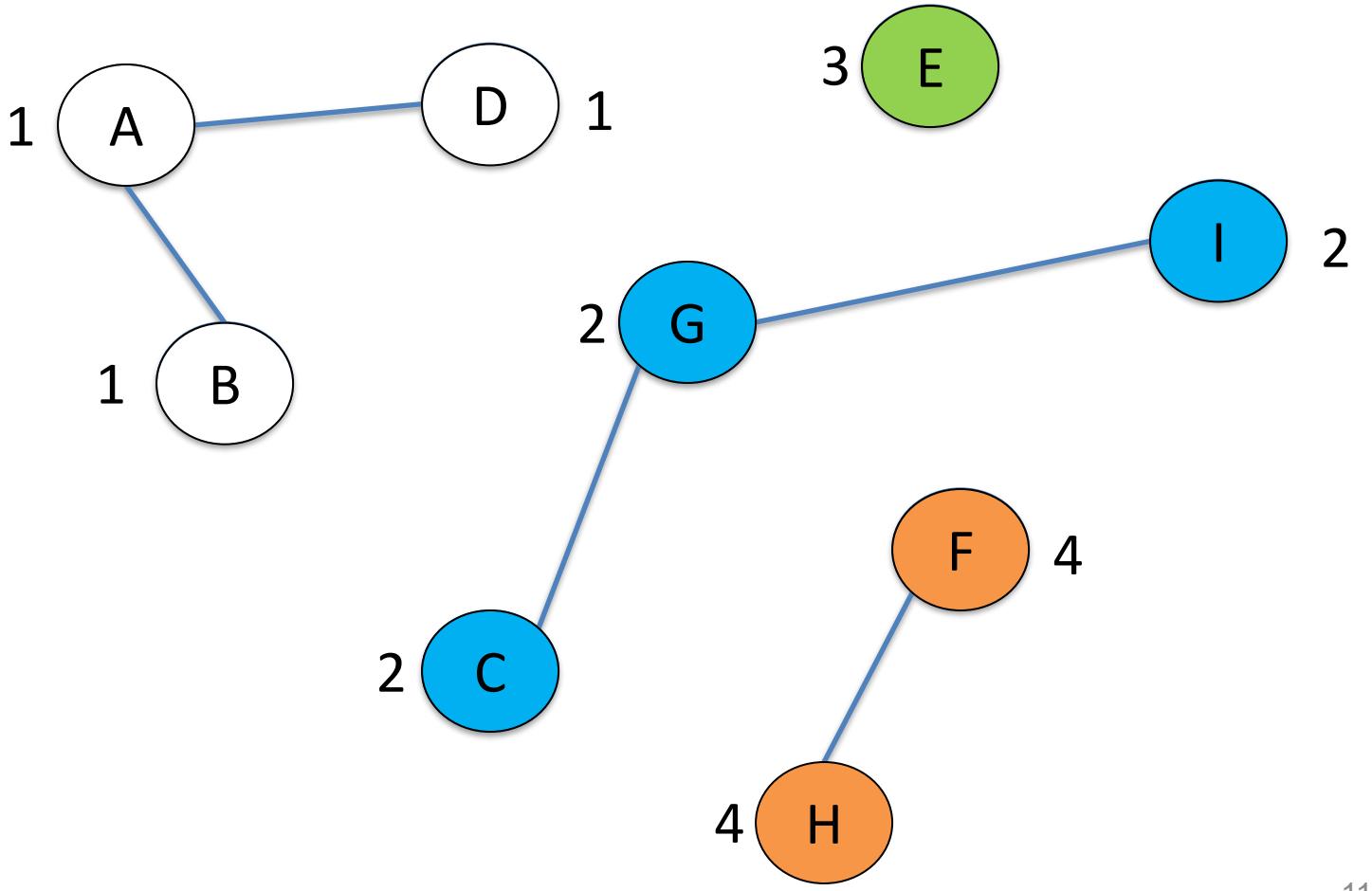
Application 2: finding connected components (1/2)

Algorithm:

```
label[1..n] ← 0                 # Initialize all node labels to 0 (unvisited)
id ← 1                          # Component ID counter

for u ← 1 to n:                 # Loop over all nodes
   if label[u] = 0:             # If node u is unvisited
      Q ← [u]                   # Start BFS from u
      label[u] ← id             # Assign current component ID

      while Q ≠ ∅:
         v ← Q.pop()            # Dequeue node v
         for w in neighbors(v):# Check neighbors of v
            if label[w] = 0:    # If neighbor is unvisited
               label[w] ← id    # Label it with current component ID
               Q.push(w)        # Enqueue neighbor

      id ← id + 1               # Move to next component ID
```

Application 2: finding connected components (1/2)

- Initially, each node has label 0



BFS algorithm

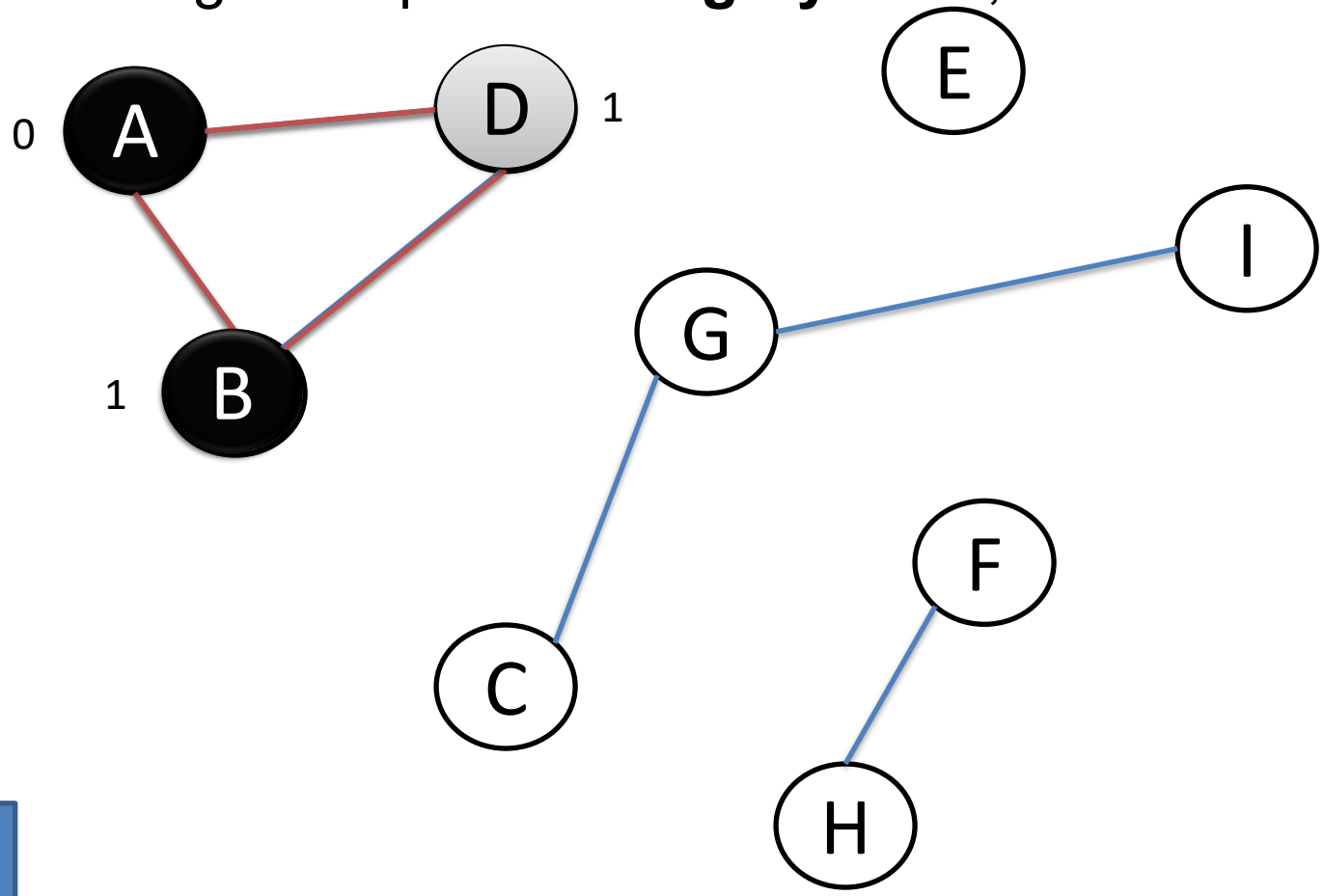
```
BFS(G, start):
    for u in G:                     # Initialize all nodes
        dist[u] ← ∞                 # Distance from start is infinite
        colour[u] ← white           # Mark all nodes unvisited

    dist[start] ← 0
    colour[start] ← gray            # Start node discovered
    Q ← [start]                     # Initialize queue with start

    while Q ≠ ∅:
        u ← Q.pop()                 # Dequeue next node
        for v in neighbors(u):      # Explore neighbors
            if colour[v] = white:   # If unvisited
                dist[v] ← dist[u] + 1   # Update distance
                colour[v] ← gray        # Mark as discovered
                Q.push(v)               # Enqueue neighbor
        colour[u] ← black           # Mark node as fully explored
```

Suppose we have a graph $G = (V, E)$ containing $|V| = n$ nodes and $|E| = m$ edges.

BFS takes $O(n + m)$ time and space.

Application 3: finding cycles

- Repeatedly do BFS from any unvisited node, until all nodes are visited.
- In any of these BFSs, if we see an edge that points to a **gray** node, then **there is a cycle**!



Enqueue node A

Dequeue node A

Enqueue node B

Enqueue node D

Dequeue node B

Try to enqueue A; black, so do nothing

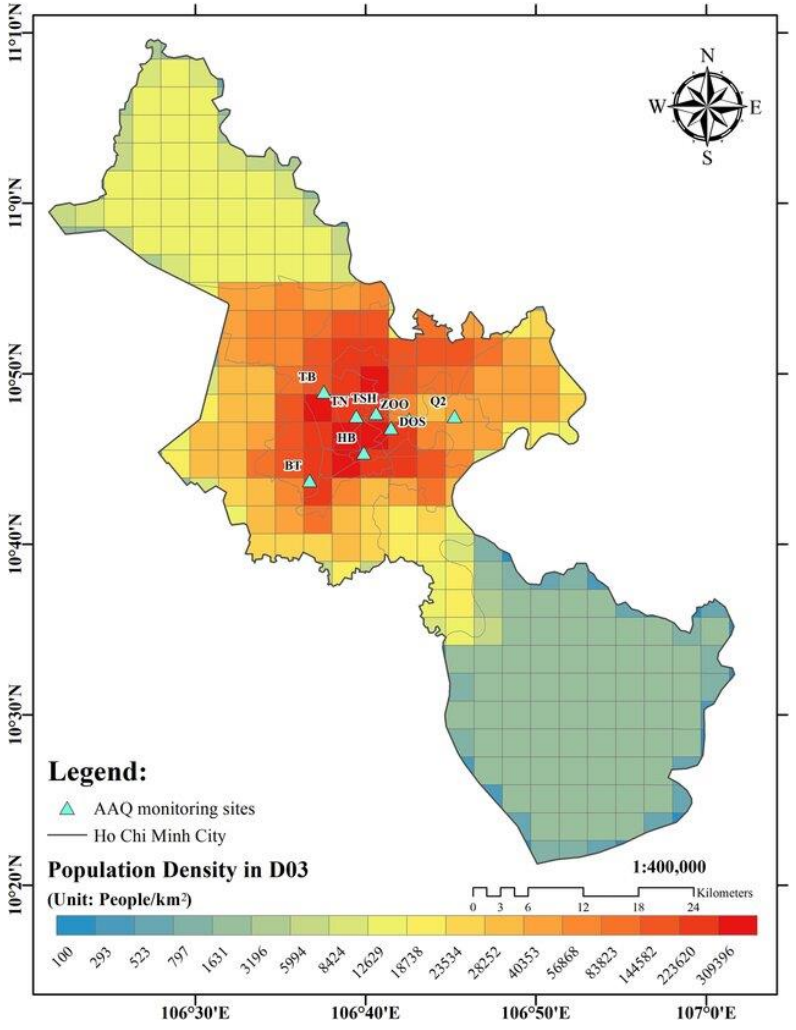
Try to enqueue D; gray node, so **cycle!**

Application 4: computing distance

- Suppose we have an $m \times n$ grid of areas.
- Initially, there is one area affected by an infectious disease!
- Each day, the virus spreads from each infected area to its nearest area (going up, down, left and right).
- How long before all area are infected?

- How can we represent this problem as a graph problem?
  - Nodes are areas.
  - There is an edge between two areas if the areas are adjacent (next to one another).

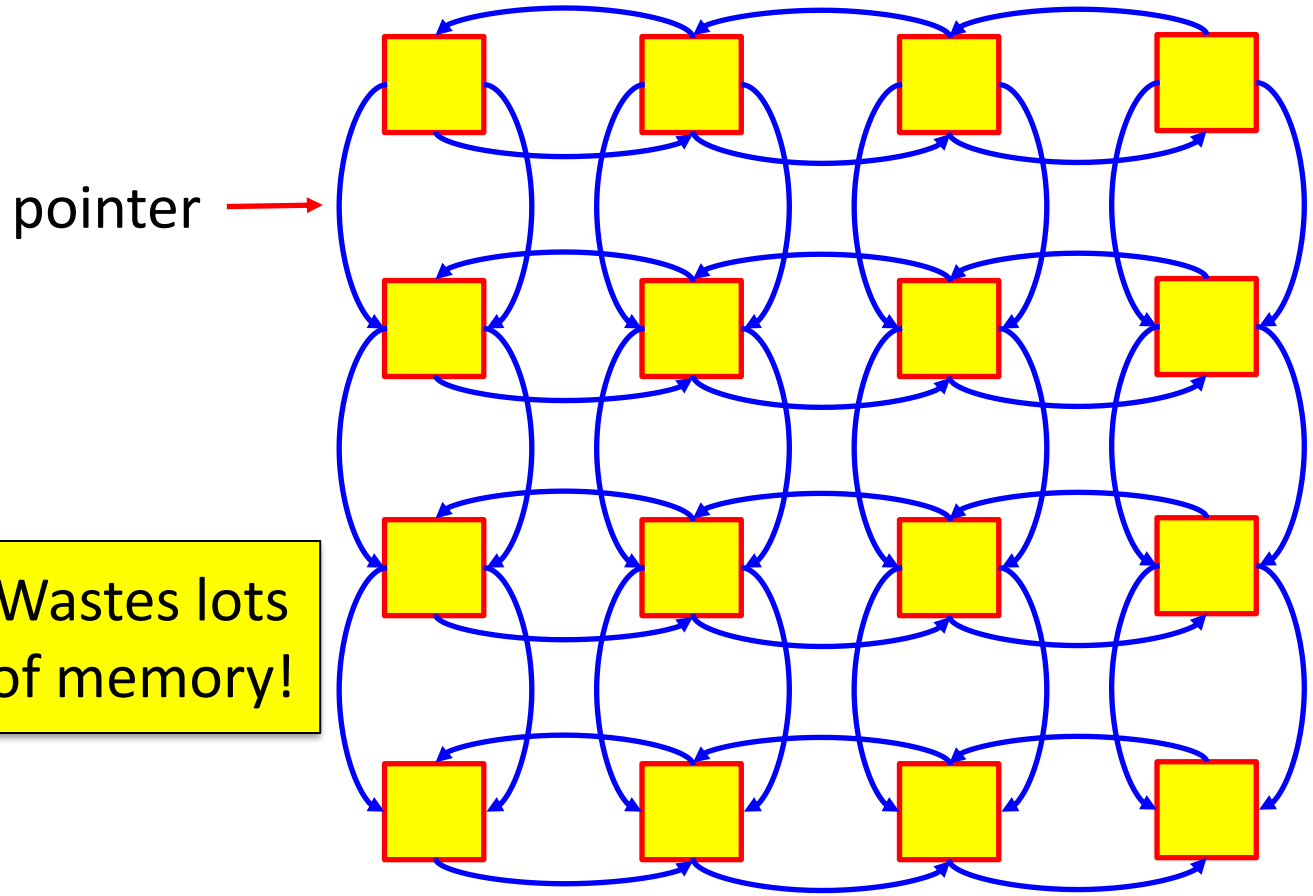
Application 4: computing distance (1/22)



Bui, Long Ta, and Phong Hoang Nguyen. "Evaluation of the annual economic costs associated with PM2. 5-based health damage—a case study in Ho Chi Minh City, Vietnam." *Air Quality, Atmosphere & Health* 16.3 (2023): 415-435.

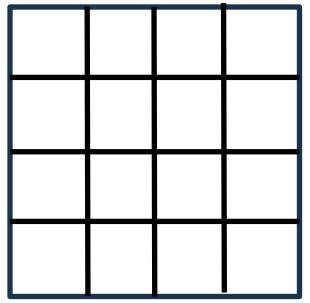
Application 4: computing distance (2/22)

- How should we store this graph in memory?
- One possibility (for a 4x4 grid of areas):



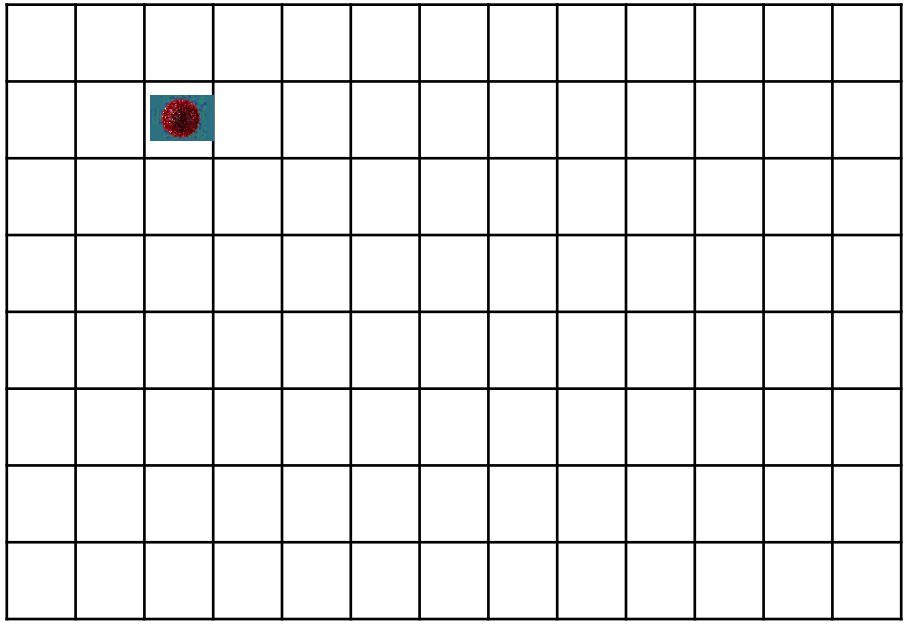
Wastes lots of memory!

- A more memory-efficient approach is to use a 4×4 array, where each cell corresponds to a specific area.



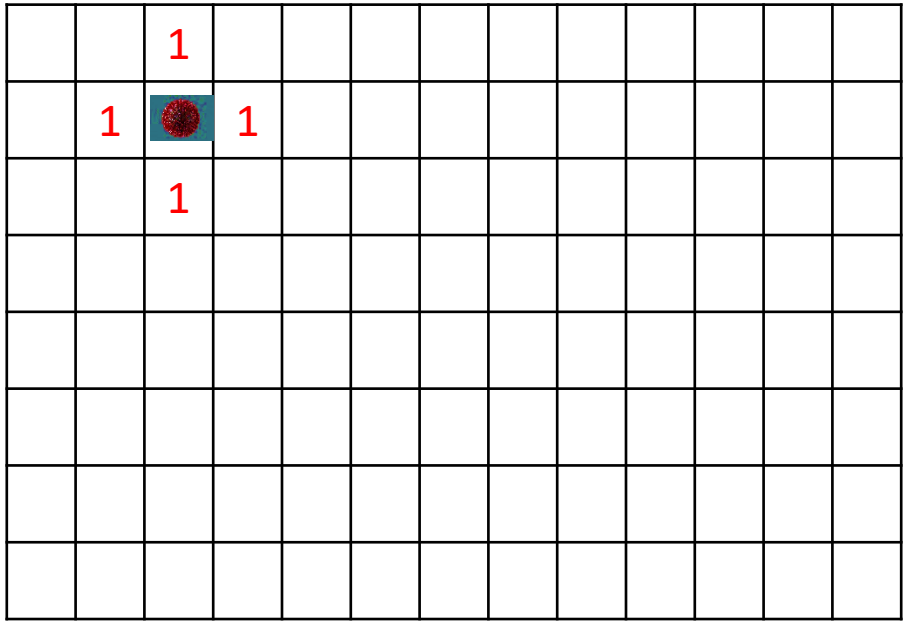
- To determine adjacency between areas, check whether their corresponding elements are adjacent (horizontally or vertically) in the array.

Application 4: computing distance (3/



Run BFS starting from the infected origin to compute the “distance” (actually time) to each area

Application 4: computing distance (4/



Application 4: computing distance (5/

	2	1	2									
2	1		1	2								
	2	1	2									
		2										

Application 4: computing distance (6/

3	2	1	2	3								
2	1		1	2	3							
3	2	1	2	3								
	3	2	3									
		3										

Application 4: computing distance (7/

3	2	1	2	3	4							
2	1		1	2	3	4						
3	2	1	2	3	4							
4	3	2	3	4								
	4	3	4									
		4										

Application 4: computing distance (8/

3	2	1	2	3	4	5						
2	1		1	2	3	4	5					
3	2	1	2	3	4	5						
4	3	2	3	4	5							
5	4	3	4	5								
	5	4	5									
		5										

Application 4: computing distance (9/

3	2	1	2	3	4	5	6					
2	1		1	2	3	4	5	6				
3	2	1	2	3	4	5	6					
4	3	2	3	4	5	6						
5	4	3	4	5	6							
6	5	4	5	6								
	6	5	6									
		6										

Application 4: computing distance (10/

3	2	1	2	3	4	5	6	7				
2	1		1	2	3	4	5	6	7			
3	2	1	2	3	4	5	6	7				
4	3	2	3	4	5	6	7					
5	4	3	4	5	6	7						
6	5	4	5	6	7							
7	6	5	6	7								
	7	6	7									

Application 4: computing distance (11/

3	2	1	2	3	4	5	6	7	8			
2	1		1	2	3	4	5	6	7	8		
3	2	1	2	3	4	5	6	7	8			
4	3	2	3	4	5	6	7	8				
5	4	3	4	5	6	7	8					
6	5	4	5	6	7	8						
7	6	5	6	7	8							
8	7	6	7	8								

Application 4: computing distance (12/

3	2	1	2	3	4	5	6	7	8	9		
2	1		1	2	3	4	5	6	7	8	9	
3	2	1	2	3	4	5	6	7	8	9		
4	3	2	3	4	5	6	7	8	9			
5	4	3	4	5	6	7	8	9				
6	5	4	5	6	7	8	9					
7	6	5	6	7	8	9						
8	7	6	7	8	9							

Application 4: computing distance (13/

3	2	1	2	3	4	5	6	7	8	9	10	
2	1		1	2	3	4	5	6	7	8	9	10
3	2	1	2	3	4	5	6	7	8	9	10	
4	3	2	3	4	5	6	7	8	9	10		
5	4	3	4	5	6	7	8	9	10			
6	5	4	5	6	7	8	9	10				
7	6	5	6	7	8	9	10					
8	7	6	7	8	9	10						

Application 4: computing distance (13/

3	2	1	2	3	4	5	6	7	8	9	10	11
2	1		1	2	3	4	5	6	7	8	9	10
3	2	1	2	3	4	5	6	7	8	9	10	11
4	3	2	3	4	5	6	7	8	9	10	11	
5	4	3	4	5	6	7	8	9	10	11		
6	5	4	5	6	7	8	9	10	11			
7	6	5	6	7	8	9	10	11				
8	7	6	7	8	9	10	11					

Application 4: computing distance (13/

3	2	1	2	3	4	5	6	7	8	9	10	11
2	1		1	2	3	4	5	6	7	8	9	10
3	2	1	2	3	4	5	6	7	8	9	10	11
4	3	2	3	4	5	6	7	8	9	10	11	12
5	4	3	4	5	6	7	8	9	10	11	12	
6	5	4	5	6	7	8	9	10	11	12		
7	6	5	6	7	8	9	10	11	12			
8	7	6	7	8	9	10	11	12				

Application 4: computing distance (13/

3	2	1	2	3	4	5	6	7	8	9	10	11
2	1		1	2	3	4	5	6	7	8	9	10
3	2	1	2	3	4	5	6	7	8	9	10	11
4	3	2	3	4	5	6	7	8	9	10	11	12
5	4	3	4	5	6	7	8	9	10	11	12	13
6	5	4	5	6	7	8	9	10	11	12	13	
7	6	5	6	7	8	9	10	11	12	13		
8	7	6	7	8	9	10	11	12	13			

Application 4: computing distance (13/

3	2	1	2	3	4	5	6	7	8	9	10	11
2	1		1	2	3	4	5	6	7	8	9	10
3	2	1	2	3	4	5	6	7	8	9	10	11
4	3	2	3	4	5	6	7	8	9	10	11	12
5	4	3	4	5	6	7	8	9	10	11	12	13
6	5	4	5	6	7	8	9	10	11	12	13	14
7	6	5	6	7	8	9	10	11	12	13	14	
8	7	6	7	8	9	10	11	12	13	14		

Application 4: computing distance (13/

3	2	1	2	3	4	5	6	7	8	9	10	11
2	1		1	2	3	4	5	6	7	8	9	10
3	2	1	2	3	4	5	6	7	8	9	10	11
4	3	2	3	4	5	6	7	8	9	10	11	12
5	4	3	4	5	6	7	8	9	10	11	12	13
6	5	4	5	6	7	8	9	10	11	12	13	14
7	6	5	6	7	8	9	10	11	12	13	14	15
8	7	6	7	8	9	10	11	12	13	14	15	

Application 4: computing distance (13/

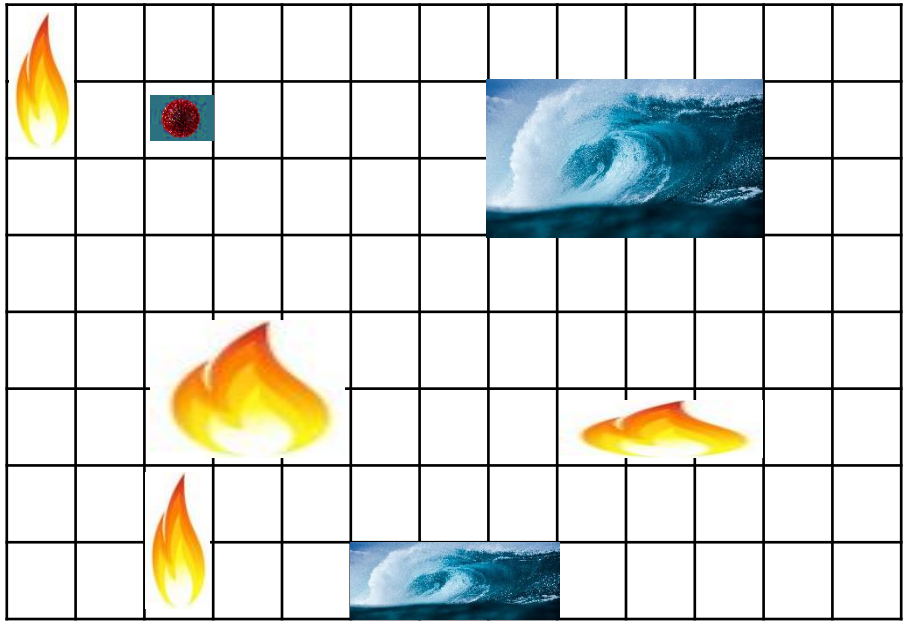
3	2	1	2	3	4	5	6	7	8	9	10	11
2	1		1	2	3	4	5	6	7	8	9	10
3	2	1	2	3	4	5	6	7	8	9	10	11
4	3	2	3	4	5	6	7	8	9	10	11	12
5	4	3	4	5	6	7	8	9	10	11	12	13
6	5	4	5	6	7	8	9	10	11	12	13	14
7	6	5	6	7	8	9	10	11	12	13	14	15
8	7	6	7	8	9	10	11	12	13	14	15	16

16 hours until all areas are infected!

average time = 7,28 hours

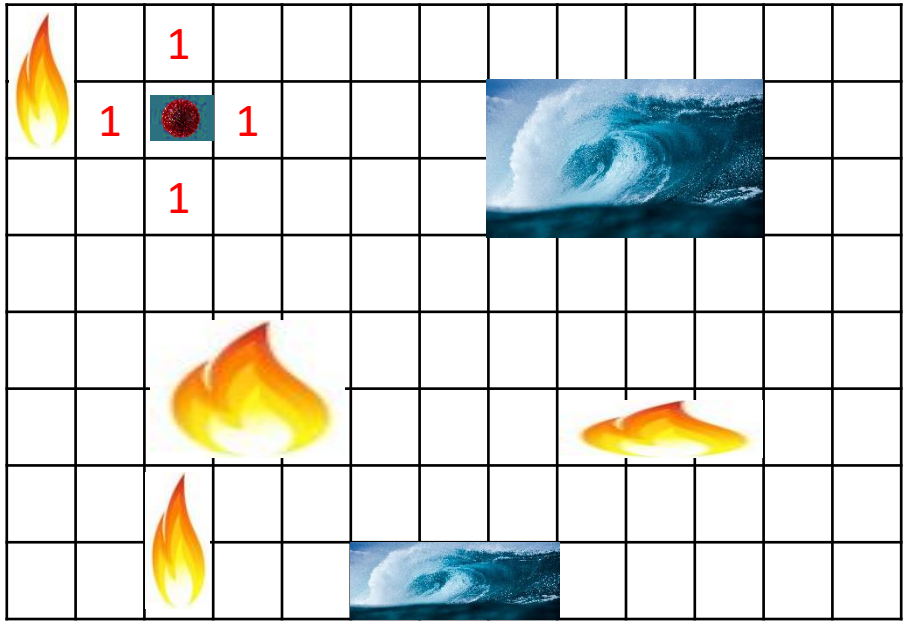
Application 4 (modified): computing distance with fires and oceans (1/17)

What if there are fires and oceans in the grid, where viruses cannot pass (bc of being killed)?

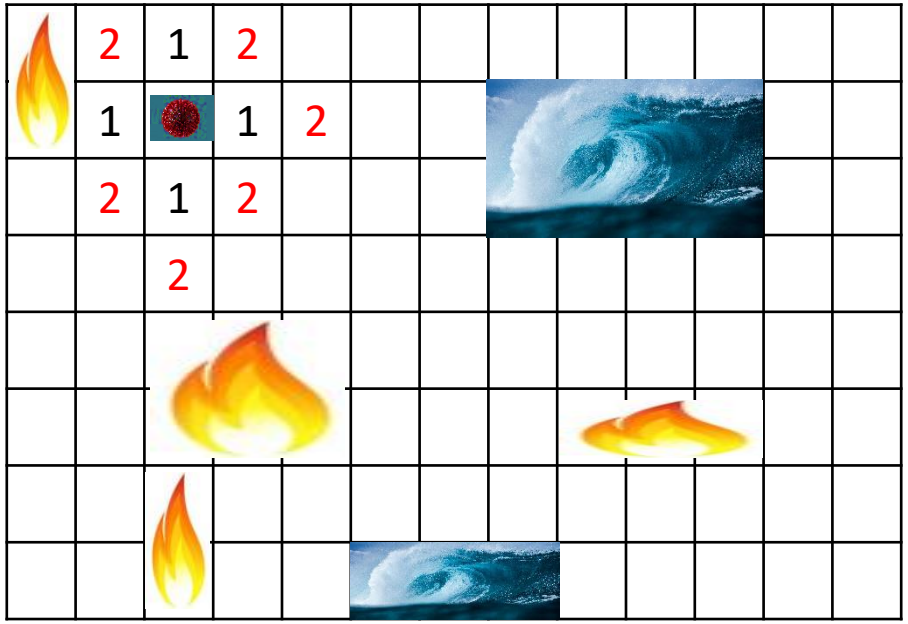


Just don’t let BFS visit the fires and oceans!

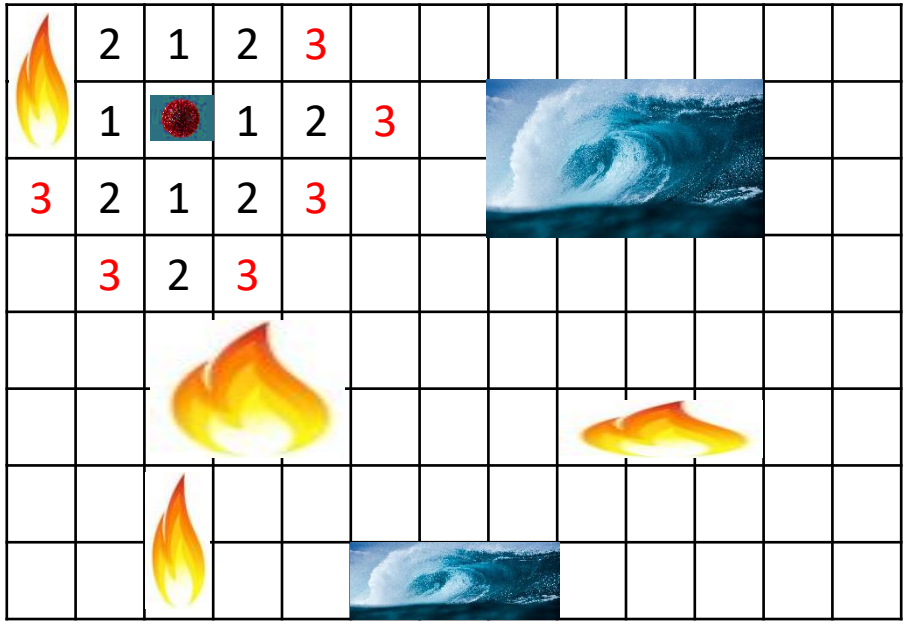
Application 4 (modified): computing distance with fires and oceans (2/17)



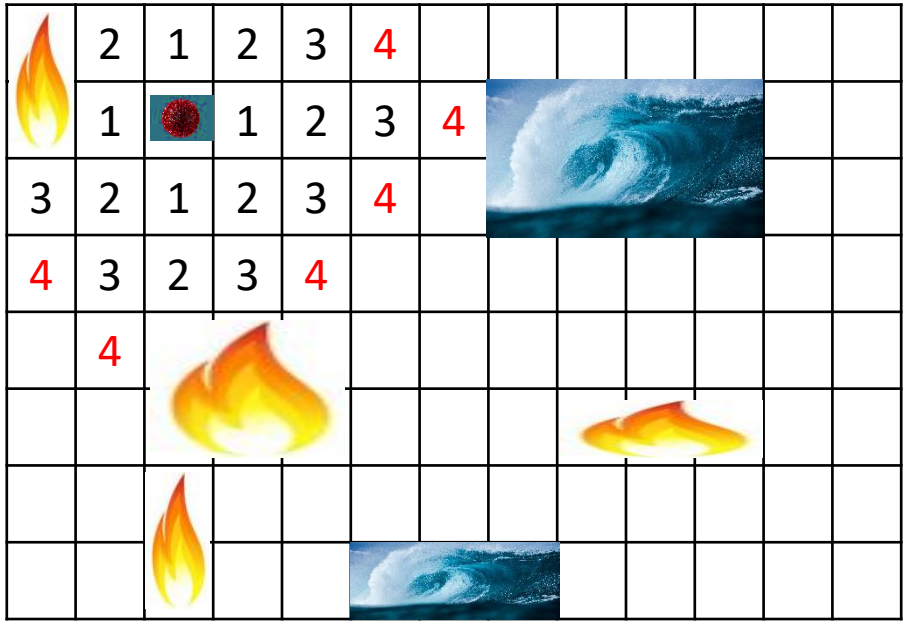
Application 4 (modified): computing distance with fires and oceans (3/17)



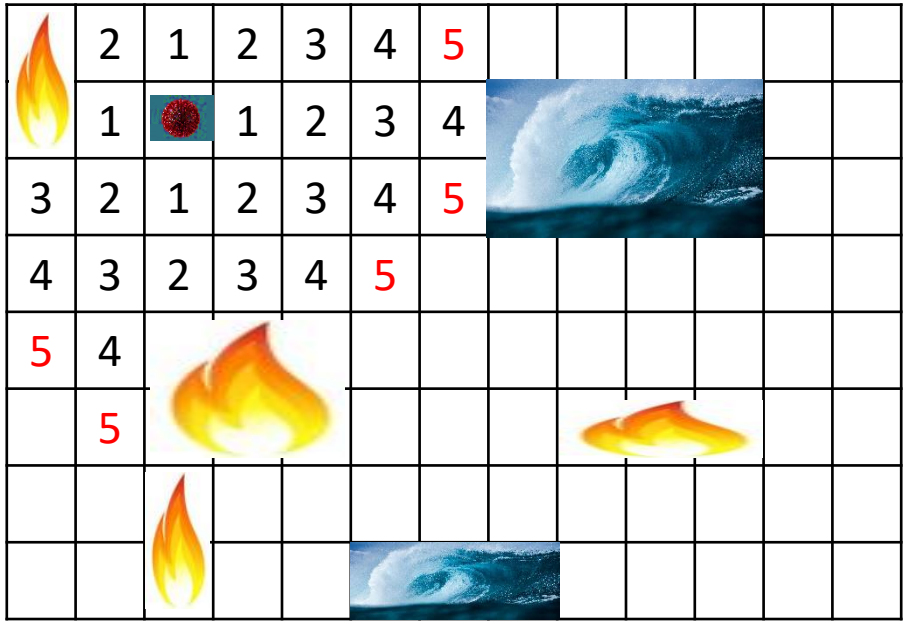
Application 4 (modified): computing distance with fires and oceans (4/17)



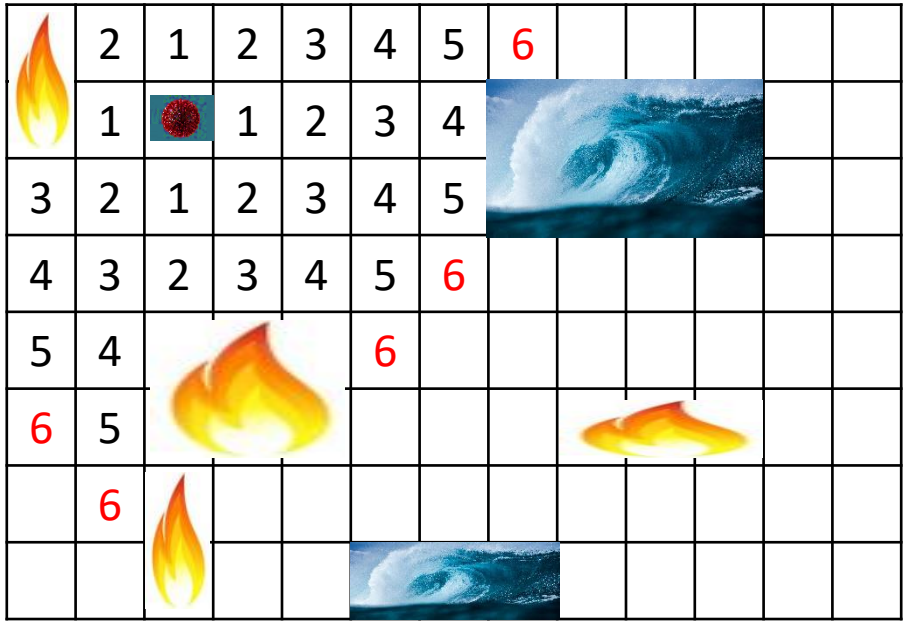
Application 4 (modified): computing distance with fires and oceans (5/17)



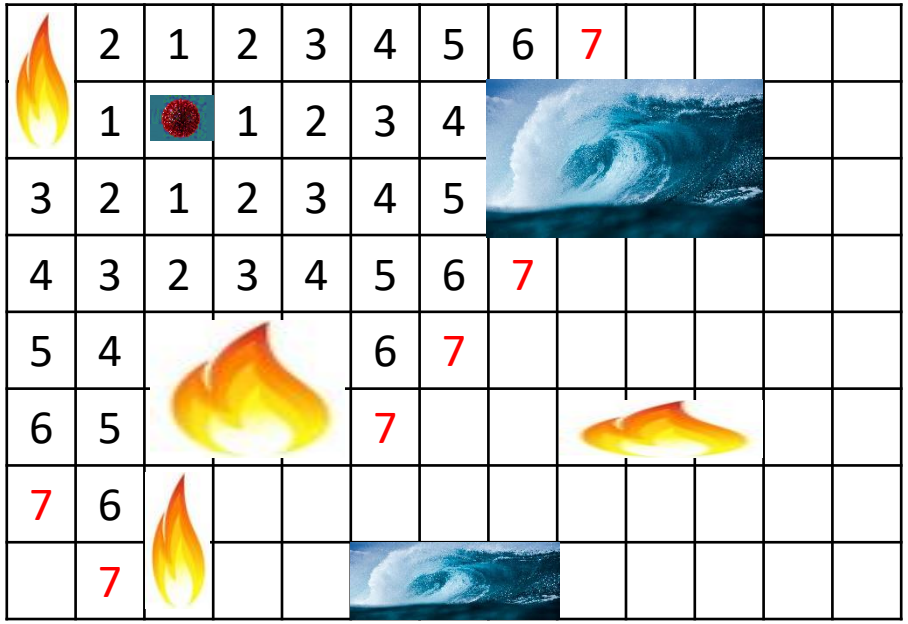
Application 4 (modified): computing distance with fires and oceans (6/17)



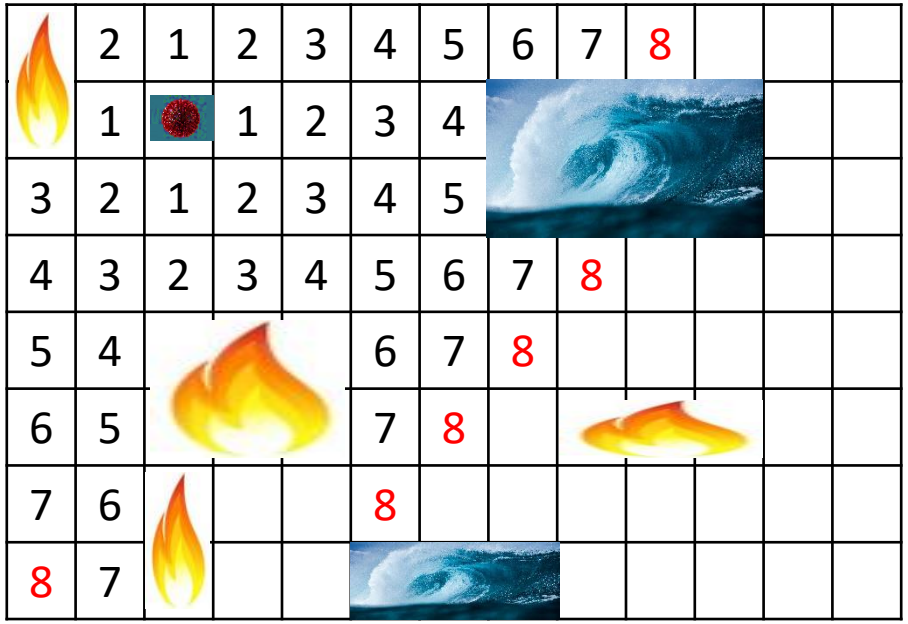
Application 4 (modified): computing distance with fires and oceans (7/17)



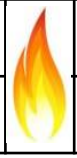
Application 4 (modified): computing distance with fires and oceans (8/17)



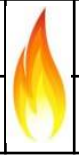
Application 4 (modified): computing distance with fires and oceans (9/17)



Application 4 (modified): computing distance with fires and oceans (10/17)

	2	1	2	3	4	5	6	7	8	9		
	1		1	2	3	4						
3	2	1	2	3	4	5						
4	3	2	3	4	5	6	7	8	9			
5	4				6	7	8	9				
6	5				7	8	9					
7	6			9	8	9						
8	7											

Application 4 (modified): computing distance with fires and oceans (11/17)

	2	1	2	3	4	5	6	7	8	9	10	
	1		1	2	3	4						
3	2	1	2	3	4	5						
4	3	2	3	4	5	6	7	8	9	10		
5	4				6	7	8	9	10			
6	5				7	8	9					
7	6		10	9	8	9	10					
8	7			10								

Application 4 (modified): computing distance with fires and oceans (12/17)

	2	1	2	3	4	5	6	7	8	9	10	11
	1		1	2	3	4					11	
3	2	1	2	3	4	5						
4	3	2	3	4	5	6	7	8	9	10	11	
5	4				6	7	8	9	10	11		
6	5				7	8	9					
7	6		10	9	8	9	10	11				
8	7		11	10								

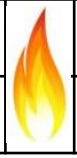
Application 4 (modified): computing distance with fires and oceans (13/17)

	2	1	2	3	4	5	6	7	8	9	10	11
	1		1	2	3	4					11	12
3	2	1	2	3	4	5					12	
4	3	2	3	4	5	6	7	8	9	10	11	12
5	4				6	7	8	9	10	11	12	
6	5				7	8	9					
7	6		10	9	8	9	10	11	12			
8	7		11	10				12				

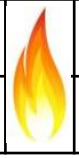
Application 4 (modified): computing distance with fires and oceans (14/17)

	2	1	2	3	4	5	6	7	8	9	10	11
	1		1	2	3	4					11	12
3	2	1	2	3	4	5					12	13
4	3	2	3	4	5	6	7	8	9	10	11	12
5	4				6	7	8	9	10	11	12	13
6	5				7	8	9				13	
7	6		10	9	8	9	10	11	12	13		
8	7		11	10				12	13			

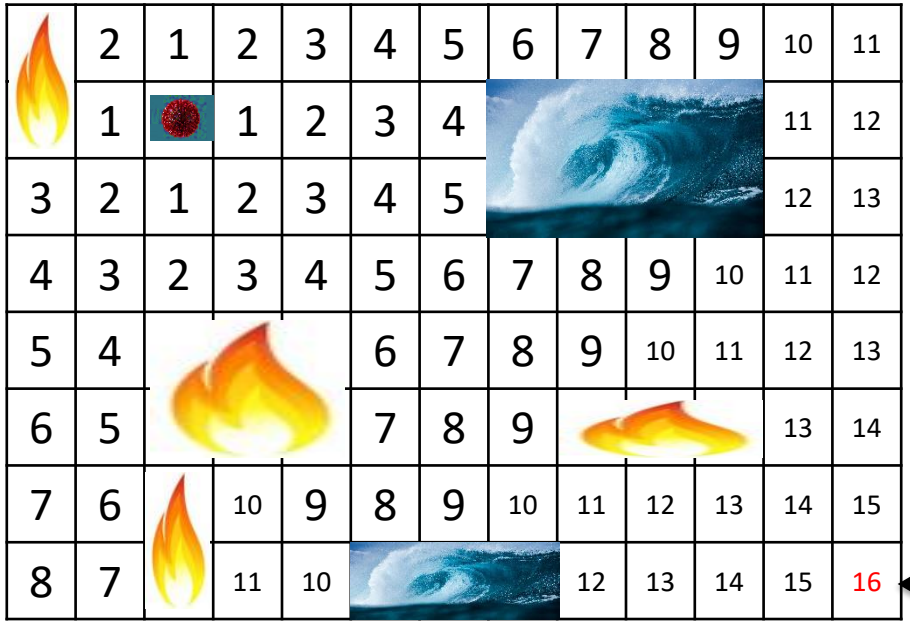
Application 4 (modified): computing distance with fires and oceans (15/17)

	2	1	2	3	4	5	6	7	8	9	10	11
	1		1	2	3	4					11	12
3	2	1	2	3	4	5					12	13
4	3	2	3	4	5	6	7	8	9	10	11	12
5	4				6	7	8	9	10	11	12	13
6	5				7	8	9				13	14
7	6		10	9	8	9	10	11	12	13	14	
8	7		11	10			12	13	14			

Application 4 (modified): computing distance with fires and oceans (16/17)

	2	1	2	3	4	5	6	7	8	9	10	11
	1		1	2	3	4					11	12
3	2	1	2	3	4	5					12	13
4	3	2	3	4	5	6	7	8	9	10	11	12
5	4				6	7	8	9	10	11	12	13
6	5				7	8	9				13	14
7	6		10	9	8	9	10	11	12	13	14	15
8	7		11	10			12	13	14	15		

Application 4 (modified): computing distance with fires and oceans (17/17)

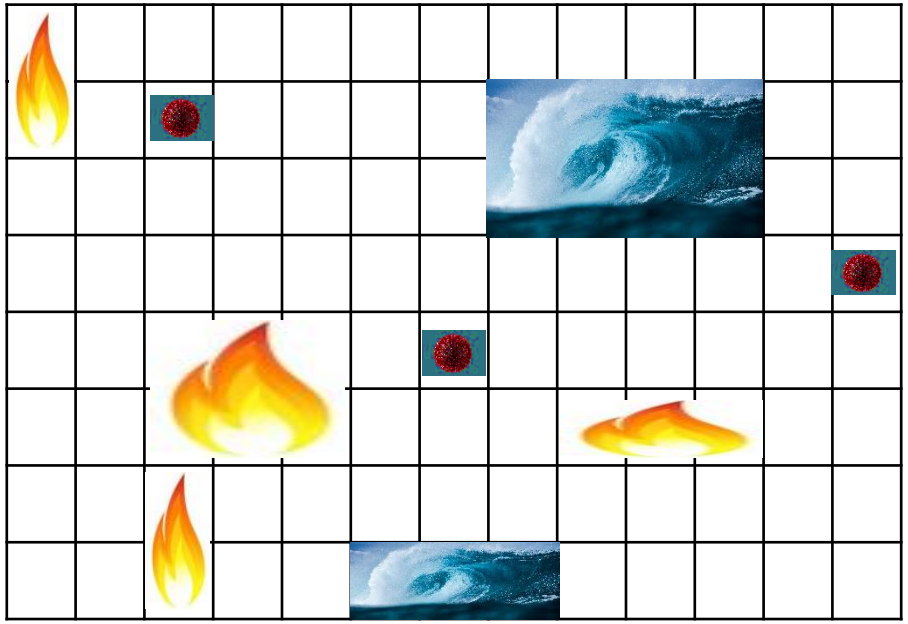


average time = 7,55 hours

> 7,28 hours when no obstacles encountered

Application 4 (modified 2): computing distance with multiple viruses (1/2)

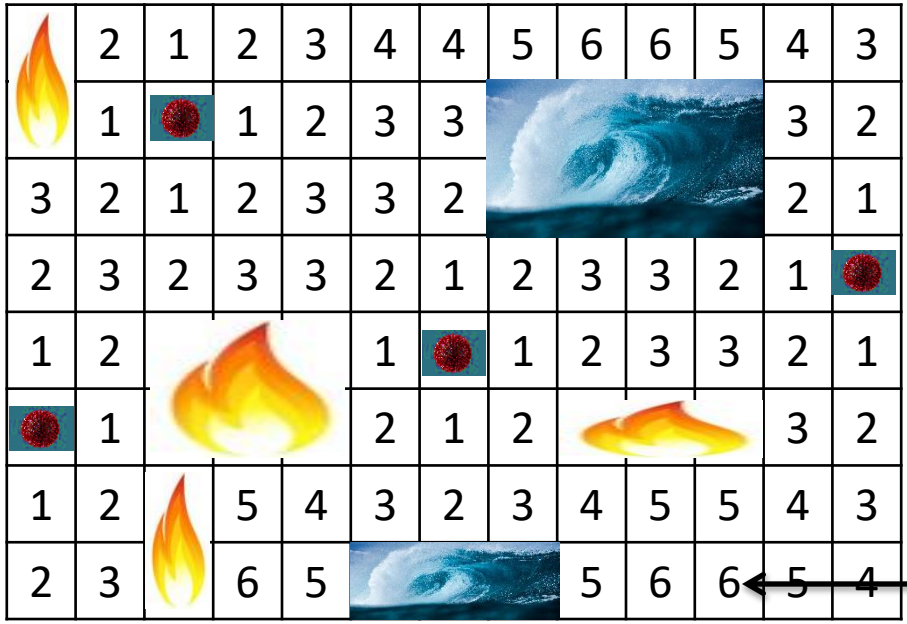
What if there are many sources of viruses?



The time that an area is reached is the smallest among ALL VIRUSES

Application 4 (modified 2): computing distance with multiple viruses (2/2)

What if there are many sources of viruses?



average time = 2,8 hours

Application 4 (modified 3): other models

- What if viruses start at different times?
- What if some viruses stop spreading?
- What if viruses spread at different speeds?

Outline

1. Introduction
2. Definition and basic properties
3. Trails, paths, and circuits
4. Graph representations
5. Isomorphisms
6. Graph traversals
   - Breadth-First Search (BFS)
   - Depth-First Search (DFS)

Depth-First Search (DFS): Recursive algorithm

- Goes as far as possible from a vertex before backing up.

```
DFS(v: vertex)
{
        Mark v as visited
        for (each unvisited vertex u adjacent to v)
                DFS(u)
}
```

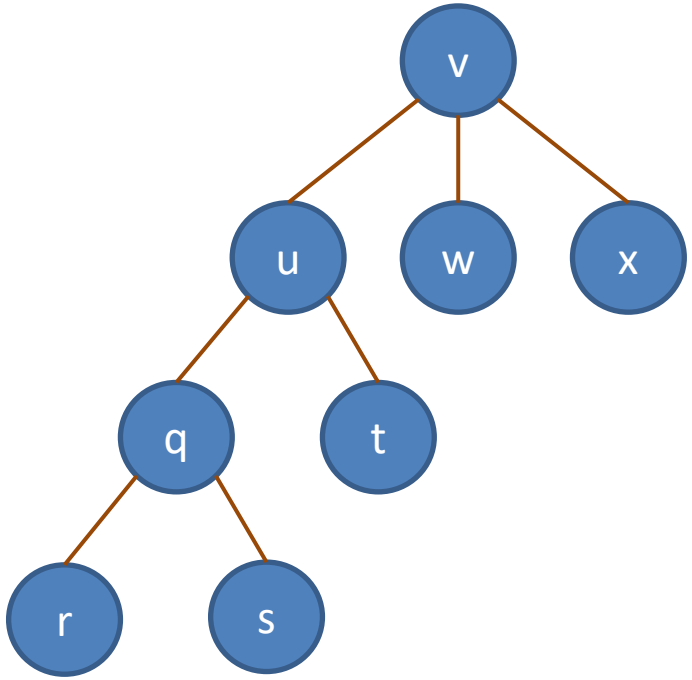
Depth-First Search (DFS): Stack-based algorithm

```
DFS(v: vertex)
        s = new empty stack
        s.push(v)
        Mark v as visited
        while (s is not empty)  {
                if (no unvisited vertices are adjacent to the vertex on the top of the stack)
                        s.pop()
                else {
                        s.push(u)
                        Marked u as visited
                }
        }
```

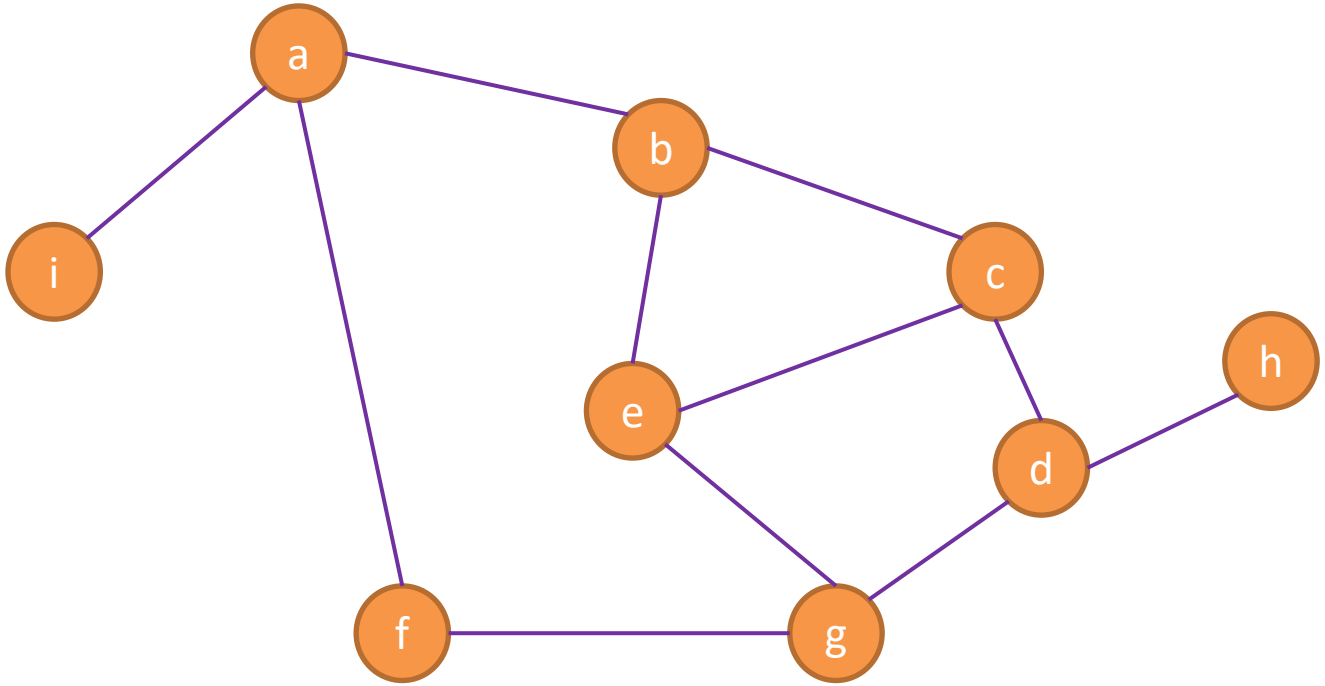
Depth-First Search (DFS): Stack-based algorithm

```
DFS(v: vertex)
        s = new empty stack
        s.push(v)
        Mark v as visited
        while (s is not empty)  {
                if (no unvisited vertices are adjacent to the vertex on the top of the stack)
                        s.pop()
                else {
                        s.push(u)
                        Marked u as visited
                }
        }
```

Depth-First Search (DFS): Example



v - u - q - r - s - t - w - x



DFS starts at ***a***:

DFS starts at ***e***:

Q & A